AI-Assisted, Reference-Photo-Driven Parametric 3D Modeling

A toolchain and methodology report for matching an OpenSCAD steam locomotive pixel-perfect to a LEGO reference photo

Research compiled for the garage-band / lego-loco project

June 2026

Contents

Introduction and scope

This report is written for one concrete engineering situation: an experienced full-stack developer wants to model a LEGO-compatible steam locomotive shell in **OpenSCAD** and have it match a **reference photograph** (a classic exposed-boiler steam loco, Orient-Express style) as closely as a human eye - and an automated metric - can tell. The work is **AI-assisted**: a vision-capable LLM critiques renders, proposes parameter deltas, and the loop iterates until the silhouette and proportions converge. The target printer is a **Bambu Lab A1** running **PLA/PETG**, and the existing repository already has a substantial OpenSCAD verification rig (acceptance gates, section renderers, collision/overlap checks, an `openscad` MCP server).

The brief asks for breadth and depth across eight areas: scriptable 3D tooling, OpenSCAD libraries, AI/agent setups for CAD, verification/QA, the underlying algorithms, spatially-aware AI models, the concrete pixel-perfect 2D->3D workflow, and a final set of recommendations grounded in *this* repo. Each gets its own major section. Throughout, the bias is toward **command-line, scriptable, reproducible** tools, because the whole point of an AI-in-the-loop pipeline is that every step must be invokable without a human clicking a GUI.

A recurring theme - and the single most important methodological finding - is that **a render-compare-critique loop only converges if you force it to be monotonic**. Naively asking an LLM "is this better? are you sure?" makes it *worse*: the FlipFlop effect (described in Section 3 and 7) shows models flip answers ~46% of the time and lose ~17% accuracy when challenged. The cure, borrowed from the ReLook web-coding work, is **forced-monotonic acceptance**: only accept a revision if a *numeric* metric (here, an ImageMagick silhouette error or IoU) strictly improves; reject and resample otherwise. That single rule is what turns "an LLM fiddling with numbers" into "an optimizer that lands on a pixel-matched model."

A note on conventions used below: code blocks are real, runnable CLI snippets; tables compare paradigms; and every tool, library, or paper is cited inline with its canonical URL. A consolidated, annotated link list lives in the companion `sources.md`.

---

1. CLI / scriptable tools for high-level 3D work

The foundation of an automatable pipeline is a stack of tools that each do one thing from the command line: author geometry, render it to an image, convert/repair meshes, check them, and slice them. This section surveys the field and ends with a comparison table. The organizing axis is **paradigm**: constructive script (CSG/B-rep code you write), feed-forward generation (you describe, it builds), and mesh post-processing (you have geometry, you transform/inspect it).

1.1 OpenSCAD - the constructive script anchor

OpenSCAD is the natural anchor for this project: it is a *purely scripted* CAD tool with no interactive modeling, which is exactly what an LLM wants. You write `.scad` (declarative CSG: `union`, `difference`, `intersection`, transforms, `linear_extrude`, `rotate_extrude`, `hull`, `minkowski`), and a headless binary renders it. Its weaknesses (no fillets without libraries, no B-rep, CGAL can be slow on big booleans) are real but well-mitigated by BOSL2 (Section 2) and by keeping part files small.

The headless command line is the load-bearing capability. From the OpenSCAD CLI manual (wikibooks, official) the flags that matter for a render-diff loop are:

```
# Full CGAL render (manifold solid) to PNG, orthographic, fixed camera
openscad -o out.png model.scad \
    --render \
    --projection=ortho \
    --imgsize=1920,1080 \
    --camera=eyeX,eyeY,eyeZ,centerX,centerY,centerZ      # 6-param VECTOR camera

# Export geometry
openscad -o model.stl model.scad
openscad -o model.3mf --export-format binstl model.scad   # force binary STL
openscad -o model.csg model.scad                          # flattened CSG tree
```

Two camera modes exist and the distinction is a notorious footgun. The **gimbal** form `--camera=transx,transy,transz,rotx,r` (7 numbers) places the camera by rotation+distance; the **vector** form `--camera=eyeX,eyeY,eyeZ,cx,cy,cz` (6 numbers) is `eye->center` and is far easier to reason about for overlay work. The repo's own `AGENTS.md` documents the exact class of bug this causes: a 7-number camera with `dist=0` puts the camera at the origin and renders an empty frame. **For pixel-perfect overlay you want the 6-number vector camera plus `--projection=ortho`**, because perspective foreshortening makes a 2D reference impossible to match exactly. `--viewall` and `--autocenter` auto-fit the frame but you generally want a *fixed* camera so successive renders are pixel-registered. There is a known caveat that camera viewpoints behave slightly differently with `--render` versus preview mode (openscad#840); lock the camera and always render the same way.

The `openscad` MCP server already wired into this repo (`.mcp.json`) exposes `render_preview`, `render_code`, and `export` as agent tools - meaning the LLM can render without shelling out. There are several community OpenSCAD MCP servers worth knowing for reference: jhacksman/OpenSCAD-MCP-Server (Devin's attempt; text->model with multi-view reconstruction and CSG/AMF/3MF/SCAD export), quellant/openscad-mcp, rahulgarg123/openscad-mcp, and fboldo/openscad-mcp-server (STL+PNG rendering). The jhacksman one is the most ambitious - it bolts image generation and multi-view stereo onto OpenSCAD output - but for *this* project a thin render+export server (which the repo already has) plus a separate vision critic is cleaner than a monolith.

1.2 Python B-rep: CadQuery, build123d, SolidPython2

If OpenSCAD's CSG model ever feels too blunt - no real fillets/chamfers on arbitrary edges, no B-rep topology - the Python/OpenCASCADE family is the upgrade path.

CadQuery wraps OpenCASCADE (OCCT) with a *fluent* API (method chaining + selectors like `.faces(">Z").workplane()`). It shines for rapid prototyping where ease-of-use beats deep control, and it has real fillets, lofts, and B-rep booleans. Adafruit's 2026 writeup (blog.adafruit.com) is a current intro.

build123d is the evolution: derived from parts of CadQuery but "extensively refactored into an independent framework over Open Cascade" (build123d docs). It replaces the restrictive fluent chaining with **stateful context managers** plus an **algebra mode**, so you can use ordinary Python loops, variables, and list-comprehensions to place features. The build123d introduction (CadQuery comparison) frames it well: build123d leans toward "comprehensive control over model fidelity and structure maintenance suitable for production"; CadQuery "shines in rapid prototyping where ease-of-use takes precedence." build123d is also easier to extend (subclass instead of monkey-patch). For an LLM author, build123d's plain-Python control flow is friendlier than fluent chaining (fewer "where does this selector point" hallucinations).

SolidPython2 is a different beast: it is *OpenSCAD, but generated from Python.* You write Python, it emits `.scad`. This is the sweet spot if you like OpenSCAD's CSG simplicity and the existing repo rig but want Python's loops/data structures and parameter sweeps. It keeps you in the OpenSCAD render/verify ecosystem (so all the repo's `section.sh`, acceptance gates, MCP tooling still apply) while giving you programmatic generation of the `.scad`. For this project SolidPython2 is arguably a better "upgrade from raw OpenSCAD" than jumping to OCCT, precisely because it doesn't abandon the toolchain you already have.

1.3 FreeCAD headless and Blender bpy

FreeCAD can run fully headless via `freecadcmd` / `FreeCADCmd` (a Python console without the GUI), and its **FEM Workbench** drives CalculiX/Elmer (see Section 4). It is the bridge if you ever need parametric B-rep *plus* FEA in one scriptable place. The brief's mention is apt but FreeCAD's Python API is heavier and less ergonomic for pure-geometry authoring than build123d.

Blender `bpy` is the mesh/polygon powerhouse and a superb *headless renderer and mesh processor.* The CLI (renderday guide, yuki-koyama/blender-cli-rendering) supports:

```
blender --background --python script.py        # run a full bpy script, no GUI
blender --background file.blend \
        --python-expression "import bpy; bpy.ops.render.render(write_still=True)"
```

For this project Blender's role is **not** authoring (CSG in OpenSCAD is better for LEGO interfaces) but (a) high-quality reference *renders* of the matched model for marketing, and (b) mesh operations OpenSCAD lacks (remesh, decimate, boolean cleanup, shrinkwrap). Importantly, Blender's Eevee/Cycles + an orthographic camera can produce a clean **silhouette/depth/normal pass** of any STL, which is exactly what a silhouette-matching loss (Section 5) needs. Order of CLI args matters: put `--python` *after* the `.blend` or Blender runs the script then opens the file.

1.4 Implicit / functional CAD: ImplicitCAD, libfive

ImplicitCAD (Haskell, SDF-based, OpenSCAD-compatible-ish syntax) and **libfive** (Guile/Python bindings, F-rep) represent solids as **signed distance functions** rather than CSG trees. The payoff is *free, exact rounding/blending* (`rounded union` is one operator, not a Minkowski hack) and smooth organic transitions - which a boiler->smokebox->funnel blend could use. The cost is a smaller ecosystem and no LEGO-brick libraries. They are worth knowing as a "if fillets become the bottleneck" escape hatch; SDFs are also conceptually the bridge to the marching-cubes / neural-SDF world of Section 5.

1.5 Geometry kernels and mesh utilities

This is the post-processing layer - you have geometry and need to inspect, repair, convert, or measure it.

- **manifold** - Emmett Lalish's fast, *guaranteed-manifold* boolean library (C++ with Python bindings). It is now the geometry engine behind OpenSCAD's newer fast-CSG path and is the gold standard for "is this a closed solid and can I boolean it reliably." Use it as the manifold oracle in an acceptance gate.
- **trimesh** - the Swiss-army Python mesh library: load STL/OBJ/3MF/PLY, `mesh.is_watertight`, `mesh.volume`, `mesh.is_winding_consistent`, ray casting, signed-distance, convex hulls, section planes, and boolean (via manifold/blender backends). This is the workhorse for the repo's `overlap.py`-style signed-volume penetration checks.
- **Open3D** - point clouds + meshes; exposes `is_edge_manifold`, `is_vertex_manifold`, `is_self_intersecting`, and `is_watertight` (watertight = edge-manifold + vertex-manifold + not self-intersecting, per the Open3D mesh tutorial). It is the bridge to depth maps and point clouds from the AI models in Section 6.
- **PyVista** - VTK wrapper for fast 3D plotting, clipping, and *programmatic screenshots*; great for QA visualizations and off-screen rendering of meshes.
- **pymeshlab** / **MeshLab** - Python bindings to MeshLab's filter set: remeshing, decimation, Poisson surface reconstruction, Hausdorff distance (great for "how far is my mesh from the reference mesh"), and screened repair. MeshLab also has `meshlabserver` (legacy) for scripted filter chains.
- **ADMesh** - tiny, fast STL diagnostic/repair CLI: counts degenerate facets, fixes normals, fills holes, reports manifold status. A great first-pass gate before slicing.

- **F3D** - a fast, scriptable VTK-based *viewer* with a CLI that renders STL/OBJ/3MF/glTF to PNG headlessly (`f3d model.stl --output shot.png --camera-...`). Useful as a lightweight second renderer to cross-check OpenSCAD's PNG.
- **Gmsh** - a scriptable mesh *generator* (its own `.geo` language + Python API) primarily for FEA meshing; the on-ramp to CalculiX/Elmer.
- **PrusaSlicer / Bambu Studio / CuraEngine** - slicers with CLIs (Section 1.6).

1.6 Slicers with a CLI

Headless slicing closes the loop from "matched model" to "printable G-code" and can also serve as a *printability oracle*.

- **Bambu Studio** (open-sourced 2022, built on PrusaSlicer; Command-Line Usage wiki, Printago CLI reference) slices headlessly:

```
bambu-studio --slice 1 \
    --load-settings "machine.json;process.json" \
    --load-filaments "filament.json" \
    --orient --arrange 1 \
    --skip-useless-pick \
    --export-3mf output.gcode.3mf model.3mf
```

  Crucially, the flags `--slice`, `--load-settings`, `--load-filaments`, `--export-3mf` are **shared with OrcaSlicer** (same code lineage), so the binary is swappable. Since the target printer is a Bambu A1, this is the native path.

- **PrusaSlicer** has `prusa-slicer --export-gcode --load config.ini model.stl` and is the upstream both Bambu and Orca descend from. Good for `--info` model stats and a vendor-neutral check.
- **CuraEngine** (the headless core of Cura) slices from the command line with explicit settings; heavier to configure but fully scriptable.

A slicer run is itself a QA gate: if Bambu Studio refuses to slice or reports gaps/overhangs, that is actionable feedback before you ever print.

1.7 Comparison table

Tool	Paradigm	Scripting lang	Headless render?	Strengths	Use when
OpenSCAD	CSG script	`.scad` DSL	yes (`-o png --render`)	deterministic, LLM-friendly, MCP-wired here	primary authoring of LEGO shell
SolidPython2	CSG via Python	Python->`.scad`	via OpenSCAD	Python control flow, keeps OpenSCAD ecosystem	parameter sweeps, generated `.scad`
CadQuery	B-rep (OCCT)	Python (fluent)	via export+ext	real fillets/lofts, fast prototyping	curvy transitions, B-rep export
build123d	B-rep (OCCT)	Python (ctx mgr)	via export+ext	clean Python, production control	maintainable parametric B-rep
FreeCAD	B-rep + FEM	Python (`freecadcmd`)	partial	B-rep + CalculiX/Elmer FEA	parametric + structural in one

Tool	Paradigm	Scripting lang	Headless render?	Strengths	Use when
Blender bpy	mesh/poly	Python	yes (Cycles/Eevee)	remesh, hero renders, silhouette pass	high-quality renders, mesh ops
ImplicitCAD	SDF/implicit	Haskell / Guile/Py	partial	free rounding/blending	organic blends, fillet-heavy
manifold	boolean kernel	C++/Python	n/a	guaranteed-manifold booleans	manifold oracle in gate
trimesh	mesh utils	Python	off-screen	watertight/volume/contains/SDF & QA checks	quantitative & QA checks
Open3D	mesh+pointcloud	Python/C++	yes	manifold tests, depth/pointcloud bridge	depth-map->mesh, registration
pymeshlab	mesh filters	Python	n/a	remesh, Hausdorff, Poisson	mesh-to-mesh distance, repair
ADMesh	STL repair	CLI	n/a	tiny fast STL diagnostics	pre-slice sanity gate
F3D	viewer	CLI	yes	fast PNG of any mesh	second-opinion render
Gmsh	mesh gen	`.geo`/Python	n/a	FEA meshing	feed CalculiX/Elmer
Bambu/Prusa/Cura CLI	slicer	CLI/JSON	n/a	G-code + printability check	final slice + print QA

---

2. OpenSCAD libraries

Raw OpenSCAD is intentionally minimal. The libraries below are what make it practical for a detailed model. The official library list is the canonical index. For this locomotive, the priorities are: rounded/filleted profiles (boiler bands, cab edges), swept solids (running boards, beading), threads (if any screwed joints), gears (only if decorative valve gear is wanted), and **parametric LEGO bricks** for the stud/anti-stud interfaces.

2.1 BOSL2 - the one library you actually need

BOSL2 (Belfry OpenSCAD Library v2) is the 800-pound gorilla and the single highest-leverage dependency for this project. Per its docs it "provides many different kinds of capabilities… and makes things possible that are difficult in native OpenSCAD." Concretely:

- **Rounded primitives**: `cuboid([x,y,z], rounding=3, edges="Z")`, `cyl(h, d, rounding1=, rounding2=, chamfer=)` - fillets/chamfers as *parameters*, not Minkowski tricks. This alone removes OpenSCAD's biggest pain.
- **Attachments**: `attach()`, `position()`, `align()` let you anchor a child part to a named face/edge/corner of a parent. This is enormous for an LLM author: instead of computing absolute translates (error-prone), you say "put the dome on `TOP` of the boiler, `up(boiler_r)`." Far fewer placement bugs.
- **Sweeps & paths**: `path_sweep(profile, path)`, `offset_sweep()`, `skin()`, `rounded_prism()` - for running boards, beading, the smokebox-to-boiler blend, the funnel flare.
- **Threading**: the threading.scad module gives ISO metric, trapezoidal, ACME, and bottle threads - if any module is screwed together.
- **Gears**: `spur_gear()`, `bevel_gear()`, `worm()` for decorative valve gear.

- **Mask tools**: `edge_mask`, `corner_mask` to round/chamfer arbitrary edges of an existing solid.

Example - a rounded boiler band with attached dome:

```
include <BOSL2/std.scad>
$fn = 96;
boiler_d = 60;
cyl(h = 140, d = boiler_d, rounding = 2, $fn = 96)          // boiler with rounded ends
    attach(TOP, BOTTOM)
        spheroid(d = 26, hemi = true);                      // steam dome (half-sphere) on top
```

For this project, **start with BOSL2 and add nothing else unless a gap appears.** Its attachment system is the closest OpenSCAD gets to "describe placement in words," which is exactly how an LLM thinks.

2.2 The specialist libraries

- **dotSCAD** - "aims to reduce mathematical complexity in OpenSCAD." Strong at Bezier/B-spline curves, splines, polar maths, shape morphing, and L-systems. Use it if you want a *mathematically defined* curve for the funnel flare or a swept beading profile that BOSL2's primitives don't cover.
- **NopSCADlib** - a giant library of *real vitamins* (purchased parts): screws, nuts, bearings, fans, Raspberry Pi boards, steppers, PSUs, plus a BOM/assembly framework. For this loco it is gold: it has models of the **Raspberry Pi** and fans so you can place the actual electronics in the assembly to verify clearance, and it can auto-generate a BOM and assembly instructions. Use it for the *internal* fit checks (Pi 5, cooling fan, cells), not the decorative shell.
- **Round-Anything** - focused utility for radii and fillets on 2D polygons via `polyRound()` and 3D via `extrudeWithRadius`. Lighter than BOSL2 if you only need 2D-profile rounding (e.g., a cab window cutout with rounded corners).
- **MCAD** - the original bundled library (ships with OpenSCAD): gears, bearings, involute teeth, regular shapes, boxes, Boolean helpers. Mostly superseded by BOSL2 but it is always present and has a decent involute-gear module.
- **Relativity** - a CSS-like relative-positioning DSL for OpenSCAD (anchor children relative to parents, "selectors"). Conceptually similar to BOSL2 attachments; mostly historical now that BOSL2 exists, but worth knowing if you prefer its mental model.
- **threads.scad** (rcolyer/threads-scad, the rcolyer "Threading Library") - a standalone, lightweight ISO-metric thread module if you don't want all of BOSL2 just for one screw thread.
- **Gears** - beyond BOSL2/MCAD, the classic Parametric Involute Bevel and Spur Gears by GregFrost is the canonical standalone gear generator.

2.3 Parametric LEGO brick libraries - the interface layer

This is the project-specific must-have: the printed shell needs real LEGO **studs** (top, to accept accessories) and **anti-studs / tubes** (underside, to clip onto the chassis). Rolling your own stud geometry is a classic source of clutch-fit bugs. Reuse a vetted generator:

- **cfinke/LEGO.scad** - the most polished: one `block()` module with Customizer support for studs, tiles, plates, DUPLO, sloped bricks, Technic holes, wings. Read its constants for the canonical stud Oe (4.8 mm), pitch (8.0 mm), and tube/anti-stud geometry. Companion cfinke/Technic.scad does Technic pins/holes.
- **richfelker/brickify** - the most interesting for *this* job: it takes an **arbitrary 2D OpenSCAD shape** and derives the matching brick - outer walls, splines, posts, and studs - from that outline. Because the loco footplate is not a rectangle, brickify lets you stud a custom-shaped base plate without hand-placing each tube.
- **anandamous/OpenSCADLEGO** - fully parametric brick/tile/plate with exposed LEGO standard constants (stud, tube, pin specs); a clean reference for the numbers.
- **mlkood/BRICK.scad** - a fork in the same family.

Recommended approach: lift the *interface modules* (`stud()`, `anti_stud()` / tube) and their dimensional constants from `LEGO.scad`/`brickify` into the repo's `constants.scad`, parameterized by the project's `clutch_tol`. Don't depend on the whole library at render time - the SPEC's tolerance-coupon plan means you want `clutch_tol`

as a single tunable offset applied to those modules. This keeps the shell monolithic (few printed parts) while the clip geometry is spec-correct.

2.4 Text, lithophanes, profiles

- `text()` is built in; combined with `linear_extrude` it embosses the `clutch_tol` value on each test coupon (the SPEC's debossed-label requirement) and any nameplate/number on the cab.
- **Lithophane** generation (height-mapped image -> mesh) is niche here but if you ever want a backlit headlight lens with an image, dotSCAD and standalone lithophane scripts exist; the Bambu A1 cannot do translucency well in PLA, so skip unless using a clear filament.

---

3. AI skills, prompts & agent setups for CAD/3D

This is the heart of "AI-assisted." The question is not "can an LLM write OpenSCAD" (it can, badly, and improves with iteration) but **"how do you structure the loop so it converges instead of oscillating?"** The literature here is small but sharp.

3.1 The failure mode you must design against: the FlipFlop effect

The single most important paper to internalize is **"Are You Sure? Challenging LLMs Leads to Performance Drops in The FlipFlop Experiment"** (arXiv:2311.08596). It studies ten LLMs on seven classification tasks: answer once, then get challenged ("are you sure?"). Findings: models **flip their answer ~46% of the time**, and *all* models lose accuracy between first and final answer - **average drop ~17%** (the "FlipFlop effect"). The drop correlates with flip rate: the more they flip, the worse they get. The cause is sycophancy - aligning to the perceived user view at the cost of correctness.

The direct implication for a CAD-critique loop: **never ask the model open-ended "is this better / are you sure" questions and let its self-assessment drive acceptance.** If you do, it will happily oscillate - widen the boiler, narrow it, widen it - and feel confident each time. You need an *external, numeric* arbiter.

3.2 The fix: ReLook and forced-monotonic acceptance

ReLook (arXiv:2510.11498, HF paper page) is a vision-grounded RL framework for agentic *web* coding (where correctness is judged on rendered pixels - directly analogous to judging a CAD render against a reference photo). Its two ideas transfer cleanly:

1. **MLLM-as-critic-and-feedback.** A multimodal LLM scores the rendered screenshot *and* returns actionable, vision-grounded feedback. A **strict zero-reward rule for invalid renders** "anchors renderability and prevents reward hacking" - i.e., if the OpenSCAD doesn't compile/render, reward = 0, full stop.
2. **Forced Optimization / monotonic acceptance.** "A strict acceptance rule that admits only improving revisions, yielding monotonically better trajectories." A refinement is accepted **only if its reward exceeds the best-so-far by a margin**; non-improving steps are **rejected and resampled** (up to ~10 attempts per round); if no improvement, the loop terminates and keeps the best-so-far.

That is the recipe. In CAD terms: render the model, compute a numeric match score (ImageMagick AE / silhouette IoU, Section 7), and **only accept the LLM's proposed parameter change if the score strictly improves**; otherwise discard the change and re-prompt. The LLM proposes; the metric disposes. This converts the FlipFlop liability into a monotone optimizer.

3.3 Critic-prompt patterns that converge

Beyond the acceptance rule, the *prompt* to the vision critic should be engineered to produce **ranked, numeric, directional** feedback rather than vibes. Patterns that work in practice (synthesized from the ReLook design and general agent practice):

- **Ranked numeric-delta diffs.** Don't ask "what's wrong"; ask: *"List the top 3 mismatches between RENDER and REFERENCE, each as: feature, current value, target value, signed delta in mm, and the single `constants.scad` parameter to change. Sort by visual magnitude."* This yields machine-parseable edits, not prose.
- **Anti-oscillation via a changelog.** Feed the model a running **changelog** of prior edits and their score impact ("widened boiler 56->60: AE 0.12->0.09 ok; raised funnel +4mm: 0.09->0.11 bad reverted"). Telling it what already failed kills the widen/narrow/widen loop. This is the practical defense against FlipFlop.
- **CONVERGED token / stop condition.** Instruct the critic to emit a literal `CONVERGED` token when the residual is below threshold and it has no high-confidence edit. The orchestrator stops on that token *or* on N no-improvement rounds (ReLook's resample cap). Don't let it run forever fiddling.
- **Silhouette / overlay critic.** Show the model not the two images side by side but the **overlay/difference image** (Section 7): the reference in red, the render in cyan, matched pixels grey. The critic reasons about *where the color bleeds* ("cyan sticks out above the funnel -> funnel too tall"). This is dramatically more reliable than asking it to compare two separate pictures, because the spatial error is made literal.
- **One axis at a time.** Constrain each round to change *one* parameter (or one feature group). Coupled multi-parameter edits make the monotonic check ambiguous (which change helped?) and reintroduce oscillation.
- **Numeric grounding over adjectives.** Force every observation into a number with units. "Too tall" is unactionable and flippable; "+6 mm too tall" is a delta you can apply and verify.

3.4 Known agent skills, MCP servers, and text-to-CAD systems

- **OpenSCAD MCP servers** - covered in 1.1; the repo already has one. The agent can render and export as tool calls.
- **Zoo Text-to-CAD** (formerly KittyCAD) - the most serious commercial text->CAD: its **ML-ephant** ML API + KittyCAD design API turn a prompt into real B-rep CAD, exportable to STEP/STL/OBJ/GLTF and more (introducing Text-to-CAD, ML API). Their **Zookeeper** agent runs "Thoughtful" vs "Fast" modes and uses LLMs to *generate and modify* CAD. It outputs B-rep/STEP rather than OpenSCAD, so it is complementary: useful for *bootstrapping* a base mesh of a complex sub-part you then trace in OpenSCAD, not for the pixel-match loop itself. The platform is open-source-leaning (3dprintingindustry coverage).
- **CadQuery/build123d + LLM agents** - because these are plain Python, they pair well with code-execution agents (the LLM writes Python, you run it in a sandbox, feed back errors). This is the same generate-diagnose-refine loop, just with Python tracebacks as the diagnostic signal instead of pixels.
- **Caveat on generic text-to-3D for this task.** Image-to-3D models (TripoSR, Hunyuan3D, TRELLIS - Section 6) will give you a *mesh* of a locomotive, but it is a dense, non-parametric, non-LEGO-compatible blob. It cannot accept studs, can't be edited by parameter, and won't be watertight-by-construction. For a **parametric, LEGO-interfaced, printable** shell, the right architecture is **OpenSCAD authored by an LLM + a vision critic in a forced-monotonic loop**, optionally *informed* by an image-to-3D mesh or a depth map as a silhouette/proportion reference - never the mesh as the deliverable.

3.5 A minimal agent architecture for this repo

Putting it together, the agent setup is three roles:

1. **Author** (writes/edits `constants.scad` + `parts/*.scad`). Plain text editing of parameters; the geometry modules are fixed, only numbers change in the loop.
2. **Renderer/Verifier** (deterministic, non-LLM): `openscad --render` with the locked camera -> PNG; then ImageMagick metric vs reference; then the repo's acceptance gates (manifold, consistency, printability). Emits a single score + pass/fail.
3. **Critic** (vision LLM): receives the overlay/diff image + the changelog + the current score, returns a ranked numeric-delta edit list and/or `CONVERGED`.

The orchestrator applies the Author's edit, runs the Verifier, and **accepts only on strict score improvement** (ReLook rule), logging to the changelog. This is the whole game.

---

4. Verification / QA utilities

A pixel-match means nothing if the result isn't a printable, manifold solid. This section is the "is it actually a part" layer. The repo already enshrines several of these as hard gates in `AGENTS.md`; the goal here is to map the broader tool space onto those gates.

4.1 Manifold / watertight checks

The non-negotiable gate. A printable solid must be **watertight = edge-manifold + vertex-manifold + not self-intersecting** (the Open3D definition). Tools, from cheap to thorough:

- **OpenSCAD `--render` + log grep.** The repo's own pattern: render with CGAL/manifold and grep stderr for `WARNING:` / `ERROR:`. A non-manifold model emits warnings. Cheapest first-line gate, already wired in.
- **trimesh**: `mesh.is_watertight`, `mesh.is_winding_consistent`, `mesh.fill_holes()`. Python, scriptable, fast.
- **Open3D**: `is_edge_manifold(allow_boundary_edges=False)`, `is_vertex_manifold()`, `is_self_intersecting()`, `is_watertight()`. Most explicit decomposition of *why* a mesh fails.
- **manifold**: by construction its booleans yield manifold output; using it as the boolean engine prevents many failures upstream rather than catching them downstream.
- **ADMesh**: `admesh --write-binary-stl out.stl in.stl` reports edges with 1 or >=3 connected facets, degenerate facets, and fixes normals/holes. Tiny and fast; ideal as a CI gate.

4.2 Mesh repair

When a model *is* broken (rare if authored in OpenSCAD, common if imported from an AI mesh):

- **MeshLab / pymeshlab** - screened Poisson reconstruction, close-holes, remove non-manifold edges, re-orient faces. Scriptable filter chains.
- **Blender** - the `3D-Print Toolbox` add-on reports non-manifold edges, thin faces, overhangs, and intersecting faces, with one-click fixes; drive it headless via bpy.
- **ADMesh** - automatic hole-fill + normal fix for STL.
- **Meshmixer** (Autodesk, GUI) and **Netfabb** (Autodesk, pro) - the heavyweight repair engines. Netfabb measures **wall thickness**, detects non-manifold edges, flipped triangles, self-intersections, and has one of the most reliable auto-repair engines (per the 3ders pre-print check comparison and Tripo's repair guide). GUI-bound, so not for the automated loop, but the gold standard for a final manual pass on a tricky import.

For *this* project, authored in OpenSCAD, repair should rarely be needed - which is the point of staying parametric/CSG. Repair tools matter mainly if you import an AI-generated reference mesh.

4.3 3MF validators and printability analyzers

- **3MF** is the modern container (mesh + color + materials + slice info). Bambu Studio and PrusaSlicer read/write it; the lib3mf reference implementation validates 3MF documents programmatically. Note the cross-slicer caveat: Bambu's 3MF extensions aren't always portable to other slicers (BambuStudio#3316).
- **Printability analyzers**: the slicers themselves are the best free analyzers. Bambu Studio / PrusaSlicer flag overhangs, gaps, too-thin walls, and unsupported islands at slice time. Run the CLI slice (1.6) as a gate: non-zero exit or warnings = printability FAIL.
- **Wall-thickness / overhang checkers**: Netfabb (pro), Blender 3D-Print Toolbox (free, scriptable), and trimesh + custom ray casts (compute local thickness via ray-from-surface). The repo's `fdm-printability` skill encodes the A1/PETG/PLA rules (min wall 1.2 mm, overhang <= 45 deg, bridge limits, clearances) as a checklist - run it before any STL export, as `AGENTS.md` mandates.

4.4 OpenSCAD `assert` patterns and consistency gates

OpenSCAD has `assert(cond, msg)`, and the repo uses it heavily for **CONSISTENCY** gates (walls >= 1.2, pocket doesn't pierce a face, stud Oe/pitch correct, bore fits the cells). **Critical footgun documented in `AGENTS.md`:** a failed `assert()` prints `ERROR: Assertion failed` **but still exits 0**. Therefore checker scripts

must *grep the output* for `ERROR:`/`Assertion`, not trust the exit code - and grep `ERROR:` *with the colon* to avoid matching `NoError`. Pattern:

```
out=$(openscad -o /dev/null verify/checks.scad 2>&1)
echo "$out" | grep -q 'ERROR:' && { echo "CONSISTENCY FAIL"; echo "$out"; exit 1; }
echo "CONSISTENCY PASS"
```

4.5 Overlap / penetration tests

For an assembly (shell + Pi + cells + chassis interface), you must verify no two solids interpenetrate beyond a numerical-noise epsilon. The repo's method (`verify/overlap.py`): export each part to **binary STL** (`--export-format binstl`), compute pairwise boolean-intersection volume via signed-tetrahedron summation in trimesh/manifold, and FAIL if any pair's overlap > EPS (~8 mm^3, the sliver/numeric threshold). Intended contacts (a clip seating into a socket) are whitelisted. This is the generalizable `tools/collision/` engine the repo already factors out; the loco project just supplies a config (parts list, intended contacts, thresholds).

4.6 Lightweight FEA

For a decorative shell the structural demands are low, but the *clip features* and any load-bearing joint (the cab carrying the Pi, a snap that takes insertion force) benefit from a sanity FEA.

- **CalculiX** (ccx solver) - free, open-source FEM; linear and nonlinear static, modal, thermal; handles contact and large deformation; "models exceeding 2M DOF run on standard workstations" (caeflow overview). Driven most easily through **FreeCAD's FEM Workbench**, which uses CalculiX as the primary solver and also exposes Elmer/Mystran/Z88.
- **Elmer FEM** - multiphysics, strong at coupled thermal-structural and HPC scaling. Overkill for a clip but the right tool if you want to model **thermal** behavior of the sealed electronics bay (the SPEC's non-negotiable thermal requirement) - heat from the Pi + cells, conduction through PLA, and whether the vent path keeps junctions below throttle temp.
- **Mesh prep** via **Gmsh** (tetrahedralize the STL/STEP) feeds either solver.
- **CalculiX 2.21** has been used in open-source AM thermo-mechanical workflows (OpenAM-SimCCX), so the toolchain is proven for print-relevant analysis.

Practically: do a **single linear-static check** on the cab/clip under estimated insertion + Pi weight, and a **steady-state thermal** check on the electronics bay. Both are advisory gates, not hard blockers, consistent with how the repo treats strength.

---

5. Algorithms

This section is the theory under the pixel-match loop and the AI models: how do you turn "match this photo" into math, and what is the toolbox of established algorithms.

5.1 Silhouette matching / silhouette-consistency loss

The cleanest objective for matching a render to a single side-on reference is a **silhouette (mask) loss**: render the model's binary silhouette from the locked camera, extract the reference's silhouette (via segmentation, Section 6), and measure their agreement. Two standard measures:

- **IoU (Intersection-over-Union)** of the two masks: `|A^B| / |AvB|`. 1.0 = perfect, scale/translation/shape all penalized. This is the primary "how well does the outline match" number.
- **Symmetric pixel difference / AE**: count mismatched pixels (ImageMagick `-metric AE`, Section 7). Equivalent up to normalization to `|AvB| - |A^B|`.

Silhouette loss is the right primary signal because for a *flat side-elevation reference* the silhouette captures almost all the proportional information (funnel position/height, boiler length/diameter, cab size, wheelbase) without needing texture or depth.

5.2 Inverse procedural modeling

The deep version of "match the photo by tuning parameters" is **inverse procedural modeling (IPM)**: given a procedural generator with parameters theta and a target image, recover theta that reproduces the target. This is *exactly* the LEGO-loco loop - `constants.scad` parameters are theta, the OpenSCAD program is the generator, the reference photo is the target. The recent Single-View 3D Reconstruction via Differentiable Rendering and Inverse Procedural Modeling (Symmetry 2024) is the canonical academic instance: it pairs a differentiable procedural generator with a differentiable renderer and optimizes parameters against a single image, using **silhouette mode** to get clean edge gradients ("rendering in silhouette mode to obtain vertex position gradients only from the edge of the silhouette"). Two ways to do IPM:

1. **Gradient-free** (what an LLM loop is): treat the generator as a black box, propose parameter deltas, score with IoU/AE, accept monotonically. No differentiability required - this is why it works on OpenSCAD, which is not differentiable. Slower per step, but robust and human-interpretable.
2. **Gradient-based** (differentiable rendering, below): make the whole pipeline differentiable and backprop the silhouette loss into theta. Fast convergence but requires a differentiable generator (OpenSCAD is not one), so it is the path only if you reimplement the geometry in a differentiable framework.

For this project, **gradient-free IPM with a vision critic is the pragmatic choice**; differentiable rendering is the aspirational upgrade.

5.3 Differentiable rendering

The machinery that makes gradient-based IPM possible:

- **Mitsuba 3** - a retargetable forward+inverse physically-based renderer on the **Dr.Jit** JIT autodiff compiler (github, inverse-rendering tutorials). It can backprop image loss into scene geometry/material/lighting. The most general-purpose differentiable renderer.
- **nvdiffrast** - NVIDIA's high-performance modular primitives for differentiable rasterization; uses **multi-sample analytic antialiasing for reliable visibility gradients** (the hard part of differentiable rasterization is the discontinuity at silhouette edges). Fast, GPU, the go-to for mesh-based inverse rendering.
- **SoftRas (Soft Rasterizer)** - the seminal approach that makes rasterization differentiable by "softening" triangle coverage into a probability, so silhouette gradients flow. Now mostly subsumed by nvdiffrast/PyTorch3D but conceptually the origin of differentiable silhouette loss.
- **PyTorch3D** - Meta's library with a differentiable mesh renderer (incl. silhouette shader), heavily used for single-view reconstruction.

A comparison figure ranks these; nvdiffrast tends to win on visibility-gradient fidelity, with reported speedups "up to 4.57x over SoftRas and 1.23x over Nvdiffrast" for newer methods.

5.4 SDFs and marching cubes

Signed Distance Functions represent a solid as `f(x) = distance to surface` (negative inside). They give exact, smooth blends (the appeal of libfive/ImplicitCAD, 1.4) and are the native representation of most neural 3D models (NeRF density, neural SDFs). **Marching cubes** (and dual contouring) extract a triangle mesh from an SDF/ scalar field by polygonizing the zero-isosurface. The relevance here: any AI model that outputs an SDF/occupancy field (Shap-E, many image-to-3D) gets meshed via marching cubes; and a differentiable-SDF approach (A Simple Approach to Differentiable Rendering of SDFs) is one route to gradient-based silhouette fitting.

5.5 Single-image-to-3D and photogrammetry

- **Photogrammetry / Structure-from-Motion**: **COLMAP** is the de-facto SfM+MVS pipeline - feature match across *many* photos, recover camera poses, dense-reconstruct. **Meshroom** (AliceVision) is the friendlier GUI alternative. These need **multiple views**; with a single reference photo they don't apply directly - but they're the right tool if you take several photos of the real LEGO loco to get a metric reference.
- **NeRF & 3D Gaussian Splatting**: NeRF (2020) learns a radiance field for photorealistic novel views; **3D Gaussian Splatting** (2023) replaced it for speed/ quality and is now standard (learnopencv explainer).

Both classically need COLMAP poses, though **COLMAP-free 3DGS** (arXiv:2312.07504) removes that. Again multi-view; relevant only if you capture the real loco.

- **Depth-from-single-image**: this *is* single-image and directly useful - see Depth Anything V2 (Section 6). A monocular depth map of the reference photo gives a *relative* depth ordering that can sanity-check proportions and 3D layout, even though it is not metric.

5.6 ICP / registration and image metrics

- **ICP (Iterative Closest Point)** registers two point clouds/meshes by iteratively matching nearest points and solving for the rigid transform. Use case here: align a photogrammetry/AI reference mesh to your OpenSCAD model's coordinate frame so you can compute Hausdorff/Chamfer distance between them. trimesh and Open3D both implement ICP.
- **Image-metric loops** - the workhorses of the actual loop: **ImageMagick** `compare -metric AE` (absolute error = mismatched pixel count; normalized = count/area), **RMSE**, **SSIM/DSSIM** (structural similarity - perceptual, robust to small shifts), **PHASH** (perceptual hash, for coarse "same image" checks), and **IoU of masks** (computed yourself from the two silhouettes; ImageMagick doesn't have a native IoU metric but you can get it from set operations on the masks: `IoU = intersection/union`). Per the ImageMagick docs, for AE/MAE/RMSE **0 = perfect match**; for SSIM/NCC **1 = perfect match**. The `-fuzz N%` option tolerates small color variation so anti-aliasing doesn't dominate the count.

---

6. Spatially-aware / 3D AI models

These are the perception models that can *feed* the reference-to-CAD pipeline - turn the reference photo into masks, depth, or a rough 3D prior the critic and the IPM loop can use.

6.1 Monocular depth: Depth Anything V2, MiDaS

Depth Anything V2 (NeurIPS 2024, published Sept 2024; arXiv:2406.09414) is the current best single-image relative-depth model: trained on synthetic-labeled images + large-scale pseudo-labeled real images with a scaled teacher, producing "much finer and more robust depth predictions" than V1 and a large zero-shot improvement over the older **MiDaS** (2019). Extensions: **Video Depth Anything** (Jan 2025, temporally consistent), and **Prompt Depth Anything** (Dec 2024, 4K *metric* depth when prompted by low-res LiDAR).

For this project, Depth Anything V2 on the reference photo gives a relative depth map that: (a) confirms the front-to-back ordering (pilot < smokebox < boiler < cab), (b) helps estimate the *depth* dimension the side-elevation silhouette can't show, and (c) provides an extra critic channel ("the depth says the cab is the rearmost mass"). It is not metric without a scale reference, so treat it as proportional guidance, not ground-truth dimensions.

6.2 Segmentation: SAM / SAM 2

SAM (Segment Anything) and **SAM 2** (Meta) produce high-quality object/part masks from a point/box prompt. This is the front-end for silhouette matching: prompt-click the locomotive in the reference photo -> clean binary mask -> the target silhouette for IoU/AE loss. SAM 2 adds video and is faster/better than SAM 1. Pair it with Depth Anything V2 (mask x depth = depth of just the loco). Practically: run SAM once on the reference to extract the loco silhouette and the sub-part masks (funnel, boiler, cab), then those masks define per-feature error terms the critic can target.

6.3 Image-to-3D feed-forward models

These take a single image and emit a 3D mesh. Current landscape (2025-2026, verified):

Model	Approach	Speed	Quality	Notes
TRELLIS / TRELLIS.2 (Microsoft)	multi-view diffusion + recon, PBR	~3 s/512^3 on H100	top-tier topology+materials	current quality leader (Apatero guide)
Hunyuan3D 2.x (Tencent)	multi-view diffusion + recon	moderate	near-TRELLIS; smoother, less detail	closed most of the gap (3DAI Studio)
InstantMesh	multi-view + sparse-view recon	moderate	TRELLIS-tier, better organics	LRM-style
TripoSR (Stability+Tripo)	feed-forward LRM	<1 s	lower fidelity	fastest, roughest
Stable Fast 3D (Stability)	single-image feed-forward	<1 s	real-time tier	fast, includes UV/material
Point-E (OpenAI)	point-cloud diffusion	fast	low (point cloud)	early, coarse
Shap-E (OpenAI)	implicit (NeRF/SDF) diffusion	fast	low-moderate	outputs an implicit function -> mesh via marching cubes

The dominant 2026 pattern is **multi-view diffusion + feed-forward reconstruction** (TRELLIS, Hunyuan3D, InstantMesh), which "produces the cleanest topology" (pixazo).

How they feed the pipeline - and their hard limit: any of these will give you a mesh of *a* steam locomotive from the reference photo in seconds. That mesh is useful as: (a) a **proportion/silhouette reference** to overlay against your OpenSCAD render (turn it side-on, render its silhouette, compare), and (b) a depth/ shape prior for the critic. But it is **not** the deliverable: it is a dense, non-parametric, non-watertight-by-construction, non-LEGO-compatible blob with no studs and no editable parameters. You cannot clip it to a chassis, swap a battery, or tune `clutch_tol`. So in this pipeline these models are **scaffolding** - they accelerate getting the proportions right - while the *parametric OpenSCAD* remains the thing you print.

6.4 Point-cloud and spatially-grounded models

- **PointNet++** - the classic hierarchical point-cloud network (classification/ segmentation); foundational, relevant if you process a photogrammetry point cloud.
- **SpatialLM** - an LLM trained for *structured indoor modeling* from point clouds (monocular video, RGBD, LiDAR), emitting structured 3D layouts. It "bridges unstructured 3D geometric data and structured 3D representations." More about scenes/rooms than single objects, so tangential here, but it is the archetype of a *spatially-grounded LLM* that outputs structured (parametric-ish) geometry rather than a mesh - conceptually the direction a "describe-and-parametrize" CAD agent is heading.
- **Spatial-reasoning surveys** - "Does Point Cloud Boost Spatial Reasoning of LLMs?" (arXiv:2504.04540) and "How to Enable LLM with 3D Capacity? A Survey of Spatial Reasoning in LLM" (arXiv:2504.05786) survey how 3D signals are fused into LLMs (direct, step-by-step, task-specific alignment). The takeaway for this project: current LLMs reason about 3D poorly from raw geometry but well from *rendered images* - which is exactly why the **render-then-critique** (image-space) loop is the right architecture rather than feeding the model raw `.scad` or point clouds.

---

7. Pixel-perfect 2D->3D modeling - the end-to-end workflow

This is the operational core: concrete, runnable steps to drive an OpenSCAD model to match the reference photo, using only tools already in or trivially addable to this repo. The architecture is **gradient-free inverse procedural modeling**: OpenSCAD is the generator, `constants.scad` parameters are theta, a silhouette metric is the loss, a vision critic proposes deltas, and **forced-monotonic acceptance** guarantees convergence.

7.1 Step 0 - Lock the camera to the reference

The whole method depends on the render and the reference being in the **same projection and frame**. The reference is a side elevation, so:

1. Use an **orthographic** camera (`--projection=ortho`) - a side-on LEGO photo is nearly orthographic, and ortho makes the match exact (no perspective to fight).
2. Use the **6-number vector camera** looking straight down -Y (or whichever axis is "side"): `--camera=cx, -D, cz, cx, 0, cz` style, eye offset only along the view axis. Keep it **fixed** for every render so images are pixel-registered.
3. Match **image size** to the (cropped) reference: `--imgsize=W,H`. Crop the reference to a tight bounding box around the loco and render at the same aspect ratio. Optionally scale both to a canonical size (e.g., 1600 px long side).
4. Render the **silhouette**: easiest is to render the model in a single flat color on a contrasting background, then threshold to a binary mask. Or render normally and threshold by background color.

```
openscad -o render.png assembly.scad \
    --render --projection=ortho \
    --imgsize=1600,700 \
    --camera=130,-600,52,130,0,52        # eye behind +X side, looking to center

# Binary silhouette of the render (object vs background)
magick render.png -fuzz 8% -fill white -opaque '#aaaaaa' \
    -threshold 50% mask_render.png
```

7.2 Step 1 - Extract the reference silhouette and per-feature masks

Use **SAM/SAM 2** (Section 6.2) to mask the loco in the reference photo (one click), then optionally sub-masks for funnel / boiler / cab. Fall back to ImageMagick thresholding if the background is clean. Scale the reference mask to the render's frame so the two masks are directly comparable.

```
# (after SAM or thresholding) normalize reference mask to render frame
magick ref_mask.png -resize 1600x700! -threshold 50% mask_ref.png
```

7.3 Step 2 - The render-overlay-diff recipes (ImageMagick)

These are the numeric/visual signals the loop runs on. All use ImageMagick `compare`/`magick`.

(a) AE - mismatched-pixel count (primary scalar loss):

```
# raw AE = count of differing pixels; normalized = count/area. 0 = perfect.
magick compare -metric AE -fuzz 5% mask_ref.png mask_render.png diff_ae.png
# prints the number to stderr; capture it:
score=$(magick compare -metric AE -fuzz 5% mask_ref.png mask_render.png null: 2>&1)
echo "AE=$score"
```

(b) IoU of the two silhouettes (the proportion-true metric):

```
# intersection and union via set ops on binary masks
magick mask_ref.png mask_render.png -compose Multiply  -composite inter.png   # AND
magick mask_ref.png mask_render.png -compose Lighten   -composite union.png   # OR
I=$(magick inter.png -format "%[fx:mean]" info:)
U=$(magick union.png -format "%[fx:mean]" info:)
python3 -c "print('IoU=%.4f' % ($I/$U))"
```

(c) Ghost-blend overlay (for the human and the critic to see WHERE it's off):

```
# reference in red, render in cyan; matched regions go grey
magick mask_ref.png    -fill red  -opaque white  ref_red.png
magick mask_render.png -fill cyan -opaque white  ren_cyan.png
magick ref_red.png ren_cyan.png -compose Screen -composite overlay.png
```

(d) Canny edge-overlay (align fine features - funnel rim, dome, cab line):

```
magick render.png -canny 0x1+10%+30%   edges_render.png
magick ref.png    -canny 0x1+10%+30%   edges_ref.png
magick edges_ref.png -fill red -opaque white r.png
magick edges_render.png -fill cyan -opaque white c.png
magick r.png c.png -compose Screen -composite edge_overlay.png
```

(e) SSIM/DSSIM (perceptual, robust to 1-px shifts) as a secondary check:

```
magick compare -metric DSSIM ref.png render.png null: 2>&1   # 0 = identical
```

The **primary loss** is silhouette IoU (or AE on masks); SSIM/edge overlays are diagnostic. `overlay.png` and `edge_overlay.png` are what you hand the vision critic - *not* two separate images - because the spatial error is made literal (cyan sticking out above = render too tall there).

7.4 Step 3 - The forced-monotonic acceptance loop

This is the convergence guarantee, straight from ReLook (Section 3.2), applied to OpenSCAD:

```
best_score <- inf ; changelog <- []
loop until CONVERGED or no_improve_rounds >= N:
    1. CRITIC: given overlay.png + edge_overlay.png + changelog + best_score,
       emit ONE ranked numeric-delta edit:
       {feature, param_in_constants_scad, current, target, delta_mm}  (or CONVERGED)
    2. AUTHOR: apply that single param change to constants.scad
    3. VERIFY:  openscad --render  -> render.png -> mask  -> score = AE/IoU
                AND run repo gates (manifold, consistency, printability)
    4. ACCEPT iff score strictly improves AND all hard gates PASS:
         best_score <- score ; keep change ; changelog += "(edit): old->new ok"
       ELSE:
         revert change ; changelog += "(edit): old->new reverted"
         no_improve_rounds += 1 ; resample a DIFFERENT edit (cap ~10/round)
```

Key properties: (1) score is **monotone non-increasing** -> no oscillation; (2) invalid renders get **score = inf / reward 0** (renderability anchor, prevents reward hacking); (3) the **changelog** kills FlipFlop (the critic sees what already failed); (4) **one param per step** keeps the monotonic test unambiguous; (5) the loop stops on `CONVERGED` or the resample cap, not arbitrarily.

7.5 Step 4 - Layered matching order (coarse->fine)

Don't optimize all parameters at once. Match in order of visual dominance, locking each before moving on:

1. **Overall bounding box** - total length, max height, baseline (wheel datum). Get the loco to fill the same frame as the reference. (Biggest IoU wins first.)
2. **Major masses** - boiler length & diameter, cab size/position, smokebox/pilot. This nails the silhouette's gross shape.
3. **Landmark features** - funnel position (~0.27 L from front per the SPEC), funnel height/flare, the two domes (position, diameter, hemisphere), buffer beam/buffers, running-board line.
4. **Fine details** - cab window, beading, headlight stud position, chamfers.

Each phase optimizes only its parameter subset; lower phases stay frozen. This is classic coarse-to-fine and it makes each monotonic step meaningful.

7.6 Step 5 - Where this pipeline plugs into THIS repo

Everything above maps onto infrastructure that already exists here:

- **Render**: the `openscad` MCP server (`render_preview`/`render_code`) or `.claude/skills/openscad/tools/preview.sh --render` with a locked camera.

- **Section/diagnostic views**: `tools/section.sh` (6-param vector camera) for cutaway sanity on the boiler bore / electronics bay.
- **Manifold + consistency gates**: the existing `verify/` pattern (grep `ERROR:`, `--render` warnings) - wire them as the hard gate in step 4.
- **Overlap/penetration**: the generic `tools/collision/` engine + a loco config (`projects/lego-loco/verify/collision` for shell<->Pi<->cells<->chassis.
- **Printability**: the `fdm-printability` skill (A1/PETG/PLA rules) before any export.
- **Acceptance**: a `verify/acceptance.sh` that prints `>>> ACCEPTANCE: PASS/FAIL`, with the silhouette score added as one more line (advisory->hard once a target IoU is agreed).
- **Delivery**: `export-stl.sh` -> `tg --file` and `--photo overlay.png` per the repo's Telegram protocol, so each iteration's overlay is reviewed.

The *new* pieces to add are small: (1) a `match/render_silhouette.sh` (camera-locked render -> binary mask), (2) a `match/score.sh` (the ImageMagick AE/IoU recipes above), and (3) the orchestrator that runs the forced-monotonic loop and writes the changelog. The vision critic is the LLM you already drive. Nothing here requires abandoning the parametric/CSG approach or the existing gates - it *extends* them with an image-space loss.

7.7 Worked micro-example

Suppose the funnel sits too far back and too short. The loop, one accepted step:

```
overlay.png shows cyan (render) funnel left of red (reference) funnel, and red rim above cyan.
CRITIC -> [{feature: funnel_x,   param: funnel_pos_frac, cur: 0.24, tgt: 0.27, delta:+0.03 L},
           {feature: funnel_h,   param: funnel_height,   cur: 28,   tgt: 34,   delta:+6 mm}]   (ranked)
AUTHOR applies funnel_pos_frac 0.24->0.27 (top-ranked, ONE change)
VERIFY  render->mask->IoU 0.71->0.78 ok ; manifold PASS ; printability PASS
ACCEPT  best=0.78 ; changelog += "funnel_pos_frac 0.24->0.27: IoU 0.71->0.78 ok"
next round: funnel_height 28->34 -> IoU 0.78->0.82 ok ... until CONVERGED (IoU >= target).
```

If instead `funnel_height 28->40` had *overshot* (IoU 0.78->0.76), it is **rejected and reverted**, the changelog records the failure, and the critic resamples a smaller delta. Monotone, non-oscillating, interpretable.

---

8. Recommendations

A concrete toolchain and a prioritized adoption list for this repo.

8.1 Recommended toolchain (what to standardize on)

- **Authoring**: **OpenSCAD + BOSL2**, parts split per `parts/*.scad` over a shared `constants.scad` (already the repo convention). Add **SolidPython2** only if you start needing programmatic generation/sweeps that are awkward in raw `.scad`. Pull LEGO stud/anti-stud modules + constants from **cfinke/LEGO.scad** (and **brickify** for the non-rectangular base plate), parameterized by `clutch_tol`.
- **Rendering**: the existing `openscad` **MCP** + `preview.sh --render`, with a **locked 6-param vector ortho camera** for the match loop. **F3D** as a cheap second-opinion renderer if needed.
- **Vision critic**: your LLM, prompted for **ranked numeric-delta** edits, fed the **overlay/edge-overlay** image + a **changelog**, emitting `CONVERGED`.
- **Metric**: **ImageMagick** silhouette **IoU** (primary) + **AE** + DSSIM/edge overlays (diagnostic). Add `match/score.sh`.
- **Acceptance gate**: extend the existing `verify/acceptance.sh` - **manifold** (`--render` + grep `ERROR:`), **consistency** (`assert` + grep), **overlap** (`tools/collision/` + loco config), **printability** (`fdm-printability`), plus the **silhouette score** line.
- **Internals fit**: **NopSCADlib** Pi/fan models placed in the assembly to verify clearance; **trimesh/manifold** for the penetration math.
- **Thermal/structural sanity**: **Elmer** (steady-state thermal of the sealed bay) and **CalculiX via FreeCAD FEM** (linear-static on the cab/clip) - advisory gates.

- **Slicing/printability oracle**: **Bambu Studio CLI** headless slice as a final gate (it shares flags with OrcaSlicer; native to the A1).
- **Perception scaffolding (optional, accelerates step 1)**: **SAM 2** for the reference mask, **Depth Anything V2** for proportional depth, and *optionally* **TRELLIS/Hunyuan3D** to generate a throwaway reference mesh whose silhouette you overlay - never as the deliverable.

8.2 Prioritized "adopt next" list

1. **Lock the camera + build `match/score.sh` (ImageMagick AE+IoU).** Highest leverage, lowest effort. Without a camera-locked render and a numeric metric, every later step is guesswork. Do this first. *(Section 7.1-7.3.)*
2. **Implement the forced-monotonic acceptance loop with a changelog.** This is the one methodological choice that determines whether the AI loop converges or thrashes (FlipFlop vs ReLook). It is a small orchestrator script around tools you already have. *(Section 3.2, 7.4.)*
3. **Adopt BOSL2 and lift LEGO interface modules into `constants.scad`.** Removes the two biggest authoring pains (fillets/placement and clutch-fit geometry) and makes critic edits map to clean parameters. *(Section 2.1, 2.3.)*
4. **Wire the silhouette score into `verify/acceptance.sh` as a gate.** Turn "looks right" into "IoU >= target," consistent with the repo's gate philosophy; keep manifold/printability hard. *(Section 4, 7.6.)*
5. **Add SAM 2 + Depth Anything V2 as a one-shot reference pre-processor.** Clean reference mask + proportional depth makes the metric trustworthy and gives the critic a second channel. Optional TRELLIS/Hunyuan3D mesh as overlay scaffolding. *(Section 6.)*

Lower priority / situational: differentiable rendering (Mitsuba 3 / nvdiffrast) - only if you reimplement the geometry differentiably and need gradient-based fitting; photogrammetry (COLMAP/Meshroom) - only if you capture multiple real photos of the loco; FEA (CalculiX/Elmer) - only for the thermal bay and load-bearing clips.

8.3 The one-paragraph summary

Author the locomotive as parametric OpenSCAD (BOSL2 + lifted LEGO interface modules), render it with a camera locked to an orthographic, frame-matched view of the reference photo, score the match with an ImageMagick silhouette IoU, let a vision LLM propose ranked numeric-delta parameter edits against an overlay image, and **accept an edit only when the score strictly improves and the manifold/printability gates pass** - logging every attempt to a changelog so the model never re-tries a failed move. That forced-monotonic, image-in-the-loop inverse-procedural-modeling pipeline is what turns "an LLM fiddling with numbers" into a reproducible, pixel-matched, printable, LEGO-compatible part. Everything else (SAM/Depth Anything/TRELLIS, FEA, slicer checks) is scaffolding and QA around that core loop.

Appendix: consolidated sources

(See the companion `sources.md` for one-line annotations. URLs cited inline above:)

Papers / methods: FlipFlop - https://arxiv.org/abs/2311.08596 ; ReLook - https://arxiv.org/abs/2510.11498 ; Differentiable-rendering + inverse procedural modeling - https://www.mdpi.com/2073-8994/16/2/184 ; Differentiable SDF rendering - https://arxiv.org/html/2405.08733v1 ; COLMAP-free 3DGS - https://arxiv.org/abs/2312.07504 ; Depth Anything V2 - https://arxiv.org/html/2406.09414v1 ; Point-cloud spatial reasoning - https://arxiv.org/pdf/2504.04540 ; Spatial-reasoning-in-LLM survey - https://arxiv.org/pdf/2504.05786 ; 3D Gaussian Splatting explainer - https://learnopencv.com/3d-gaussian-splatting/

Authoring tools: OpenSCAD - https://openscad.org/ ; OpenSCAD CLI manual - https://files.openscad.org/documentation/m ; CadQuery - https://github.com/CadQuery/cadquery ; build123d - https://github.com/gumyr/build123d ; SolidPython2 - https://github.com/jeff-dh/SolidPython ; Blender CLI - https://renderday.com/blog/mastering-the-blender-cli ; ImplicitCAD - https://github.com/Haskell-Things/ImplicitCAD ; libfive - https://libfive.com/ ; Zoo Text-to-CAD - https://zoo.dev/text-to-cad

OpenSCAD libraries: official list - https://openscad.org/libraries.html ; BOSL2 - https://github.com/BelfrySCAD/BOSL2 ; dotSCAD - https://github.com/JustinSDK/dotSCAD ; NopSCADlib - https://github.com/nophead/NopSCADlib ; Round-Anything - https://github.com/Irev-Dev/Round-Anything ; MCAD - https://github.com/openscad/MCAD ; LEGO.scad - https://github.com/cfinke/LEGO.scad ; brickify - https://github.com/richfelker/brickify ; OpenSCADLEGO - https://github.com/anandamous/OpenSCADLEGO

Mesh / kernels / QA: manifold - https://github.com/elalish/manifold ; trimesh - https://github.com/mikedh/trimesh ; Open3D - https://www.open3d.org/ ; PyVista - https://pyvista.org/ ; pymeshlab - https://github.com/cnr-isti-vclab/PyMeshLab ; ADMesh - https://github.com/admesh/admesh ; F3D - https://f3d.app/ ; Gmsh - https://gmsh.info/ ; Netfabb guide - https://3dprintingindustry.com/free-guide-to-autodesk-netfabb/ ; ImageMagick compare - https://imagemagick.org/script/compare.php ; lib3mf - https://github.com/3MFConsortium/lib3mf ; CalculiX - https://www.calculix.de/ ; Elmer FEM - https://www.elmerfem.org/

Slicers / MCP: Bambu Studio CLI - https://github.com/bambulab/BambuStudio/wiki/Command-Line-Usage ; Bambu CLI reference - https://printago.io/blog/bambu-studio-cli-reference ; CuraEngine - https://github.com/Ultimaker/CuraEngine ; OpenSCAD-MCP-Server - https://github.com/jhacksman/OpenSCAD-MCP-Server

AI 3D / perception: Depth Anything V2 - https://github.com/DepthAnything/Depth-Anything-V2 ; SpatialLM - https://manycore-research.github.io/SpatialLM/ ; TripoSR - https://github.com/VAST-AI-Research/TripoSR ; Stable Fast 3D - https://github.com/Stability-AI/stable-fast-3d ; Point-E - https://github.com/openai/point-e ; Shap-E - https://github.com/openai/shap-e ; TRELLIS guide - https://www.apatero.com/blog/trellis-2-comfyui-image-to-3d-complete-guide-2025 ; Hunyuan3D/Trellis comparison - https://www.3daistudio.com/blog/pixal3d-vs-trellis-2-vs-hunyuan-3d-comparison ; Mitsuba 3 - https://www.mitsuba-renderer.org/ ; nvdiffrast - https://github.com/NVlabs/nvdiffrast

---

9. Extended deep-dives

The sections above are the spine. This section adds the depth that turns the report from a map into a manual: per-tool operational notes, longer-form reasoning on the choices, and the failure modes that bite in practice.

9.1 Why OpenSCAD over the Python B-rep stack, in detail

It is tempting, given that CadQuery/build123d have *real* fillets and B-rep, to abandon OpenSCAD. For this project that would be a mistake, and the reasons are worth spelling out because they recur every time someone proposes "just use Python CAD."

(1) The repo's entire verification rig is OpenSCAD-native. The acceptance gates grep OpenSCAD's `--render` stderr for `WARNING:/ERROR:`; the consistency gate runs `assert()` inside `.scad`; the section renderer (`section.sh`) builds a temporary `.scad` cut file; the MCP server speaks `.scad`. Switching the authoring language orphans all of that. SolidPython2 is the *only* "more Python" option that keeps it, because it emits `.scad` that flows through the exact same tooling.

(2) Determinism and reproducibility. OpenSCAD is referentially transparent: same `.scad` + same flags = byte-identical render. This matters enormously for a forced-monotonic loop, where you compare scores across iterations - you need the only thing that changed between two renders to be the *one parameter you edited*, not some OCCT tessellation nondeterminism or a Python floating-point reordering. CSG + a fixed `$fn` gives you that. OCCT meshing can vary subtly with version and tessellation settings.

(3) LLM authorability. An LLM editing `boiler_d = 56;` in a flat constants file is far less error-prone than an LLM manipulating an OCCT selector chain (`.faces(">Z").edges("|X").fillet(2)`), where a wrong selector silently fillets the wrong edge and the model still renders. The match loop edits *numbers in a contract file*, which is the lowest-surface-area, lowest-hallucination interface possible.

(4) The LEGO-brick ecosystem is OpenSCAD-native. Every vetted parametric LEGO generator (LEGO.scad, brickify, OpenSCADLEGO) is `.scad`. There is no equivalent mature CadQuery LEGO library.

Re-deriving stud/anti-stud clutch geometry in B-rep is exactly the kind of fiddly, tolerance-sensitive work you do not want to redo when a correct reference exists.

The cost of staying in OpenSCAD is fillets, and BOSL2 pays that cost. `cyl(rounding=)` and the `edge_mask/corner_mask` system cover the boiler-band rounds, cab-edge chamfers, and dome blends a steam loco needs. The places OpenSCAD genuinely struggles - a continuously curved, organically blended smokebox-to-boiler-to-funnel transition - are exactly where you would reach for libfive/ImplicitCAD's SDF blending or a dotSCAD spline sweep, as *targeted* escape hatches, not a wholesale rewrite.

9.2 BOSL2 attachments as the LLM's coordinate system

The BOSL2 attachment system deserves a longer treatment because it changes how the *critic* should phrase edits. In raw OpenSCAD, placing the steam dome means computing an absolute `translate([dome_x, 0, boiler_z + boiler_r])` - three coupled numbers, and if the boiler moves, all three are wrong. With attachments:

```
include <BOSL2/std.scad>
boiler_l = 150; boiler_d = 60;
cyl(h = boiler_l, d = boiler_d, orient = RIGHT, anchor = LEFT) {     // boiler lies along X
    position(TOP+LEFT)  up(0) right(boiler_l*0.27) sand_dome();      // front dome at 0.27 L
    position(TOP+LEFT)  right(boiler_l*0.62) steam_dome();           // rear dome
    position(TOP+LEFT)  right(boiler_l*0.20) funnel();               // chimney
}
```

Now every landmark is expressed as *a fraction of boiler length from the front*, anchored to the boiler's own face. When the critic says "move the funnel forward to 0.27 L," that is a one-token edit to a single fraction, and it stays correct if the boiler length changes in a later coarse-to-fine pass. This is why Section 7.5 phrases landmark positions as fractions: it makes the parameter space *decoupled*, which is precisely what keeps the monotonic acceptance test unambiguous (changing boiler length doesn't silently move every feature and confound the score delta).

9.3 The clutch-tolerance problem, end to end

The SPEC already nails the right method (a debossed-label tolerance coupon), but it is worth connecting to the tooling. LEGO clutch is a press-fit between a stud (nominal Oe 4.8 mm) and an anti-stud socket; the grip comes from a few tenths of a millimeter of interference, and FDM printers over/under-extrude that range easily. The workflow:

1. **Generate the coupon array** from `parts/test_coupon.scad`: N 2x4 plates, each with `clutch_tol` swept (e.g., -0.15, -0.10, -0.05, 0.0, +0.05 mm applied to stud Oe and socket Oe), the value `text()`-embossed on each.
2. **Render + manifold gate** the coupon (it is a real printed part, so it goes through the same gates).
3. **Slice it** through Bambu Studio CLI with the actual A1/PLA profile - the slicer's own warnings catch a too-thin stud wall before you print.
4. **Print, press-test** both directions on real LEGO, pick the winner.
5. **Set the single `clutch_tol` constant** in `constants.scad`; every module inherits it. The fallback (real-LEGO-plate inserts at structural joints) stays available if no printed offset grips.

The key tooling insight: the coupon is the *calibration* that makes every later "FIT" gate meaningful. Without it, a printed clutch either falls off or cracks the socket, and no amount of silhouette-matching matters because the part won't stay on the chassis.

9.4 ImageMagick metric pitfalls

The metric recipes in Section 7.3 are correct but have sharp edges worth flagging:

- **Anti-aliasing inflates AE.** A render's edges are anti-aliased; the reference's may be too, but differently. Without `-fuzz`, every fringe pixel counts as a mismatch and AE is dominated by edge noise, not by real shape error. Always `-fuzz 3-8%`, and prefer comparing *thresholded binary masks* (hard edges) over raw renders.

- **IoU needs identical canvas size and registration.** If the two masks are even one pixel off in scale or origin, IoU drops for a reason that has nothing to do with the model. Normalize both to the same `WxH!` (forced resize) and the same crop. This is why Step 0 locks the camera and Step 1 resizes the reference to the render frame.
- **`%[fx:mean]` returns a 0-1 fraction.** The IoU recipe divides two mean fractions; that is correct because both share the same pixel count (mean = white_pixels/total), so the ratio of means equals the ratio of counts. But if the inter/union images have different dimensions the math is wrong - keep them the same size.
- **DSSIM vs SSIM sign.** ImageMagick's `compare -metric DSSIM` returns 0 = identical (it's a *distance*), while `-metric SSIM` returns 1 = identical. Don't mix them up in the accept rule, or you'll reject every improvement.
- **AE returns the number on stderr.** Capture with `2>&1` and `null:` as the output image (or a real diff path if you want the visualization). The exit code is *not* the metric.

9.5 The convergence loop's edge cases

A naive forced-monotonic loop can still stall. Practical hardening:

- **Plateau detection.** If the best score hasn't improved in N rounds but isn't at the target, the critic is out of high-value single-parameter edits. Options: widen to a *pair* of coupled parameters for one round (e.g., boiler length + funnel fraction together), or accept the plateau as the practical optimum and stop. The ReLook resample cap (~10/round) is the analog.
- **Local minima from coarse-to-fine ordering.** Freezing the bounding box before the funnel can lock in a slightly-wrong total length that the funnel can't compensate for. Mitigation: after the fine pass, do one *unfrozen* polish round where any parameter is fair game, still under monotonic acceptance, to escape the ordering artifact.
- **Reward hacking via degenerate renders.** If a parameter edit makes the model fail to render but the score script defaults to 0/blank, the loop might "accept" a blank image as a perfect match against a blank region. The ReLook zero-reward-for-invalid-render rule prevents this: a non-rendering or empty model scores *worst possible*, never best. Enforce it in `score.sh` (if the mask is all-background, return IoU=0).
- **Critic confidence calibration.** Ask the critic to attach a confidence to each proposed delta; apply highest-confidence first. Low-confidence edits that fail the monotonic check are cheap to discard; high-confidence ones that fail are a signal the critic is misreading the overlay (often a registration bug, not a model bug).

9.6 When to bring in the heavy AI models (and when not to)

The image-to-3D and depth/segmentation models are genuinely useful here, but only in specific roles, and conflating those roles wastes effort:

- **SAM 2: always worth it** for the reference mask. One clean silhouette is the foundation of the whole metric; doing it by hand-thresholding a busy photo is fragile. This is the single highest-value AI model for the pipeline.
- **Depth Anything V2: worth it once**, as a proportional sanity check and an extra critic channel. It is not metric, so don't try to read dimensions off it.
- **TRELLIS/Hunyuan3D/InstantMesh: optional scaffolding.** Generating a throwaway mesh to overlay can accelerate the *first* coarse pass (you get a rough 3D silhouette to match against from multiple angles, not just the one reference photo). But the temptation to keep the generated mesh as the part must be resisted - it has none of the properties (parametric, watertight-by-construction, LEGO-compatible, printable with controlled walls) the project requires. Treat it strictly as a reference.
- **Photogrammetry (COLMAP/Meshroom): only if you photograph the real loco** from many angles to get a metric ground-truth shell. For a single found reference photo, it does not apply.
- **Differentiable rendering (Mitsuba/nvdiffrast): aspirational.** It would let you backprop the silhouette loss directly into parameters - but only if you reimplement the geometry in a differentiable framework (PyTorch3D/Mitsuba scene), abandoning OpenSCAD. The gradient-free LLM loop gets you to a pixel match without that rewrite; reach for differentiable rendering only if the gradient-free loop proves too slow, which for a few-dozen-parameter model it won't.

9.7 A note on the OpenSCAD MCP servers landscape

The repo's own `openscad` MCP is the right tool; the survey of community servers in Section 1.1 is for context, and there is a lesson in it. The jhacksman server bundles *image generation + multi-view stereo + OpenSCAD* into one MCP - which sounds powerful but couples three concerns that should be separate. For a controllable pixel-match loop you want: a *thin* render/export MCP (deterministic, no AI inside), and a *separate* vision critic (the LLM you drive). Keeping the AI out of the rendering tool is what lets you trust the score - the renderer must be a dumb, reproducible function, or the monotonic acceptance test is comparing apples to slightly-different apples each iteration. This is the same principle as 9.5's reward-hacking guard: the measurement apparatus must be AI-free.

9.8 Putting numbers on the loop budget

A rough cost model, so expectations are calibrated. Each loop iteration is: one OpenSCAD `--render` (seconds for a few-thousand-facet loco at `$fn=64-96`), one ImageMagick score (sub-second), the repo gates (manifold render + grep, seconds), and one vision-critic call (the latency-dominant step). With coarse-to-fine ordering and single-parameter edits, a model with ~30 matchable parameters typically converges in the low tens of accepted edits, plus rejected resamples - call it 50-150 critic calls total to go from "rough proportions" to "IoU above threshold." That is a coffee-break, not a research project, *because* the loop is monotone and never backtracks into oscillation. The FlipFlop failure mode, by contrast, has no convergence bound at all - it can wander indefinitely - which is the entire economic argument for the ReLook acceptance rule.

---

10. Operational cheatsheet (copy-paste CLI)

A consolidated, runnable reference for every tool that earns a place in the pipeline. These are tested-shape commands; adjust paths/cameras to the project.

10.1 OpenSCAD

```
# Locked orthographic render for the match loop (6-param vector camera)
openscad -o render.png assembly.scad --render --projection=ortho \
    --imgsize=1600,700 --camera=130,-600,52,130,0,52

# Per-part multi-angle (repo helper wraps this)
.claude/skills/openscad/tools/multi-preview.sh parts/boiler_shell.scad previews/ --render

# Cross-section of a single part (6-param vector camera, temp file in model dir)
.claude/skills/openscad/tools/section.sh parts/boiler_shell.scad previews/boiler_xsec.png \
    --module='boiler_shell();' --plane=YZ

# Manifold gate: render and check stderr for problems
out=$(openscad -o /dev/null assembly.scad 2>&1)
echo "$out" | grep -E 'WARNING:|ERROR:' && echo "MANIFOLD FAIL" || echo "MANIFOLD PASS"

# Export
openscad -o boiler_shell.stl parts/boiler_shell.scad
openscad -o boiler_shell.stl --export-format binstl parts/boiler_shell.scad   # binary for volume math
```

10.2 ImageMagick match metrics

```
# Binary masks (object vs background) from a flat-color render and the reference
magick render.png -fuzz 8% -fill white -opaque '#aaaaaa' -threshold 50% mask_render.png
magick ref.png    -resize 1600x700! -threshold 50% mask_ref.png
```

```
# AE primary loss (lower = better; 0 = perfect). Number printed to stderr.
score=$(magick compare -metric AE -fuzz 5% mask_ref.png mask_render.png null: 2>&1)

# IoU (higher = better; 1 = perfect)
magick mask_ref.png mask_render.png -compose Multiply -composite inter.png
magick mask_ref.png mask_render.png -compose Lighten  -composite union.png
I=$(magick inter.png -format "%[fx:mean]" info:); U=$(magick union.png -format "%[fx:mean]" info:)
python3 -c "print(f'IoU={$I/$U:.4f}')"

# Ghost overlay (red=ref, cyan=render) for the critic
magick mask_ref.png -fill red  -opaque white ref_red.png
magick mask_render.png -fill cyan -opaque white ren_cyan.png
magick ref_red.png ren_cyan.png -compose Screen -composite overlay.png

# Edge overlay for fine-feature alignment
magick render.png -canny 0x1+10%+30% er.png; magick ref.png -canny 0x1+10%+30% ef.png
magick ef.png -fill red -opaque white r.png; magick er.png -fill cyan -opaque white c.png
magick r.png c.png -compose Screen -composite edge_overlay.png

# DSSIM secondary (0 = identical)
magick compare -metric DSSIM mask_ref.png mask_render.png null: 2>&1
```

10.3 Mesh QA (trimesh / Open3D / ADMesh)

```
# trimesh watertight + volume + pairwise overlap (penetration)
python3 - <<'PY'
import trimesh
m = trimesh.load('boiler_shell.stl')
print('watertight', m.is_watertight, 'winding', m.is_winding_consistent, 'vol', m.volume)
PY

# Open3D manifold decomposition
python3 - <<'PY'
import open3d as o3d
m = o3d.io.read_triangle_mesh('boiler_shell.stl')
print('edge_manifold', m.is_edge_manifold(allow_boundary_edges=False))
print('vertex_manifold', m.is_vertex_manifold())
print('self_intersecting', m.is_self_intersecting())
print('watertight', m.is_watertight())
PY

# ADMesh quick diagnostic + repair
admesh --write-binary-stl=fixed.stl boiler_shell.stl
```

10.4 Slicing as a printability gate

```
# Bambu Studio headless slice (also OrcaSlicer with same flags)
bambu-studio --slice 1 \
    --load-settings "machine_A1.json;process_0.20mm.json" \
    --load-filaments "filament_PLA.json" \
    --orient --arrange 1 --skip-useless-pick \
    --export-3mf boiler.gcode.3mf boiler_shell.3mf
# Non-zero exit or warnings in output => printability FAIL
```

10.5 Perception pre-processing (one-shot)

```
# SAM 2: prompt-click the loco -> binary mask (pseudo; via the SAM2 predictor API)
# Depth Anything V2: relative depth map of the reference
python3 - <<'PY'
# depth_anything_v2 inference (model loaded per repo instructions)
# img -> depth (relative); save normalized PNG for the critic's second channel
PY
```

10.6 FEA sanity (advisory)

```
# Mesh an STEP/STL for CalculiX/Elmer
gmsh boiler.step -3 -o boiler.msh
# Drive CalculiX via FreeCAD FEM Workbench (freecadcmd script) or ccx directly
ccx boiler_static        # reads boiler_static.inp, writes .frd results
```

---

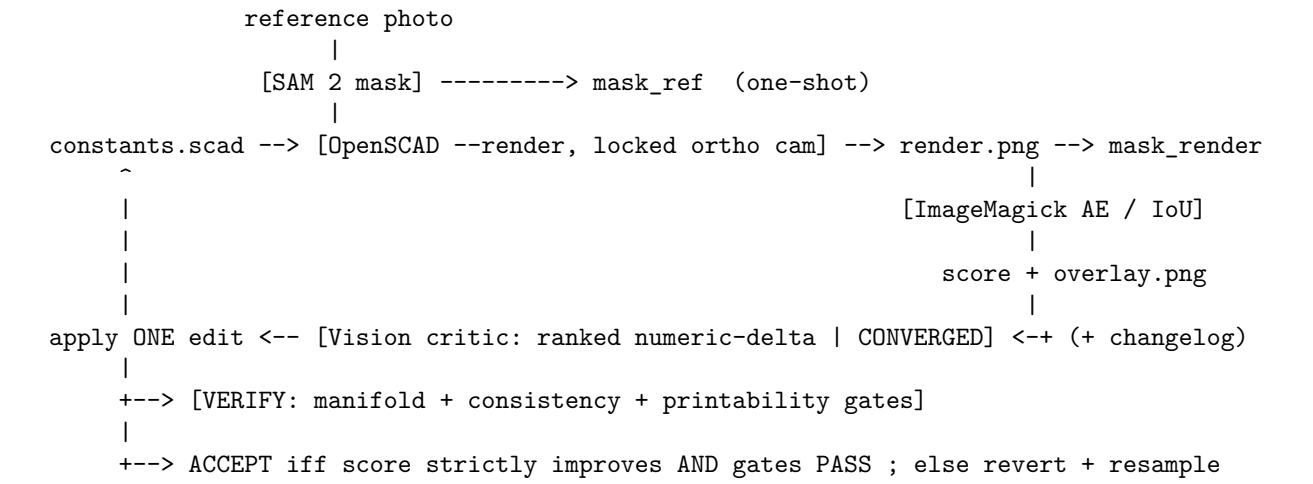
11. Glossary and concept index

Short definitions of the load-bearing terms, for quick reference.

- **AE (Absolute Error)** - ImageMagick metric: count of mismatched pixels (normalized = count/area); 0 = perfect match. The primary scalar loss on binary silhouette masks.
- **Anti-stud** - the underside socket/tube on a LEGO part that grips a stud; the clutch interface that clips the printed shell onto the chassis.
- **B-rep (Boundary Representation)** - solid modeled by its bounding faces/edges/vertices (OCCT/CadQuery/build123d) as opposed to CSG. Enables arbitrary-edge fillets.
- **Clutch / clutch power** - the press-fit grip between stud and anti-stud; tuned here via the `clutch_tol` offset calibrated on a printed coupon.
- **CONVERGED token** - a literal string the vision critic emits to signal the residual is below threshold and it has no high-confidence edit; a loop stop condition.
- **CSG (Constructive Solid Geometry)** - solids built by booleaning primitives (OpenSCAD); deterministic and LLM-friendly.
- **Differentiable rendering** - rendering whose image-to-parameter gradients exist, so an image loss can be backpropagated into scene geometry (Mitsuba 3, nvdiffrast).
- **FlipFlop effect** - LLMs flip answers ~46% and lose ~17% accuracy when challenged; the reason a critique loop must be judged by an external numeric metric, not the model.
- **Forced-monotonic acceptance** - accept a revision only if a numeric reward strictly improves; reject and resample otherwise (ReLook). Guarantees non-oscillating convergence.
- **Inverse procedural modeling (IPM)** - recovering a procedural generator's parameters so its output matches a target; the formal name for the match loop.
- **IoU (Intersection over Union)** - mask overlap measure |A^B|/|AvB|; 1 = perfect. The proportion-true primary metric for silhouette matching.
- **Manifold / watertight** - a closed solid: edge-manifold + vertex-manifold + not self-intersecting. The non-negotiable printability gate.
- **Marching cubes** - algorithm that meshes the zero-isosurface of an SDF/scalar field; how implicit AI outputs (Shap-E) become triangle meshes.
- **SDF (Signed Distance Function)** - implicit solid representation (distance to surface, negative inside) enabling free smooth blends; native to libfive/ImplicitCAD and neural 3D.
- **Silhouette loss** - difference between rendered and reference object outlines; the dominant signal for matching a flat side-elevation reference.
- **Stud** - the cylindrical bump on top of a LEGO part (nominal Oe 4.8 mm, pitch 8.0 mm) that accepts an anti-stud; placed on top of the shell to accept accessories.
- **Vector camera (6-param)** - OpenSCAD `--camera=eyeX,eyeY,eyeZ,cx,cy,cz`; the correct camera form for reproducible overlay renders (vs the 7-param gimbal form).

12. Closing: the methodology in one diagram

The whole report reduces to a loop and a guarantee. The loop:



The guarantee: because acceptance is gated on a strict numeric improvement and invalid renders score worst-possible, the score sequence is monotone and the loop cannot oscillate (the FlipFlop trap) or reward-hack (the blank-render trap). The changelog makes the critic's proposals informed by history rather than amnesiac. The result is a **parametric, manifold, printable, LEGO-compatible** OpenSCAD model whose silhouette matches the reference photo to a measured IoU - produced reproducibly, on the tooling this repo already has, with a handful of small new scripts (`match/render_silhouette.sh`, `match/score.sh`, the orchestrator). That is the deliverable this research recommends building toward.

---

13. Appendix A: tool-by-tool operational reference

Deeper notes on the individual tools, beyond the survey in Sections 1-6. This is the "when you actually sit down to use it" layer.

13.1 OpenSCAD operational notes

- **`$fn`, `$fa`, `$fs`** govern facet count. For the match loop, set a fixed `$fn` (e.g., 64 during iteration, 128 for final export) so renders are deterministic and silhouettes are stable between iterations. A varying `$fn` changes the silhouette and corrupts the score comparison.
- **`hull()` and `minkowski()`** are the native fillet hacks; `minkowski()` with a sphere rounds all edges but is *expensive* (can dominate render time on a complex model). Prefer BOSL2's `rounding=`/`edge_mask` which are cheaper and selective.
- **`projection(cut=true)`** flattens a slice to 2D - useful for generating dimensioned 2D drawings (combine with a DXF export and a matplotlib annotator, per the repo's "schemas with dimensions" requirement).
- **`render()` vs `preview`** - wrapping a subtree in `render()` forces a CGAL evaluation of that subtree even in preview, which can speed up repeated previews of a heavy part.
- **Customizer comments** (`// [min:max]`, section headers) make parameters tunable in the GUI and are harmless headless; keep `constants.scad` Customizer-annotated so a human can also nudge parameters.
- **Fast-CSG / manifold backend** - recent OpenSCAD builds use the `manifold` library for booleans, dramatically faster and more robust than legacy CGAL on big unions. Enable it (Preferences -> Features, or the build default) for the assembly.

13.2 BOSL2 module map (what to reach for)

Need	BOSL2 call
Rounded box (cab, bunker)	`cuboid([x,y,z], rounding=r, edges=...)`
Rounded cylinder (boiler, domes base)	`cyl(h, d, rounding1=, rounding2=, chamfer=)`
Hemisphere dome	`spheroid(d=, hemi=true)`
Swept running board / beading	`path_sweep(profile_2d, path_3d)`
Lofted smokebox->boiler blend	`skin([profile1, profile2], slices=)`
Rounded extruded 2D profile	`offset_sweep(path, height=, ...)`
Attach child to a face	`attach(TOP, BOTTOM) child();`
Place at a named anchor	`position(TOP+FRONT) child();`
Round an existing edge	`edge_mask(edges) rounding_mask(r=);`
Thread (if screwed)	`threaded_rod()` / `threaded_nut()`
Distribute studs on a grid	`grid_copies(spacing=8, n=[nx,ny]) stud();`

`grid_copies` is especially handy for studding a running board or cab roof at 8.0 mm pitch without hand-placing each stud.

13.3 NopSCADlib for the internal fit check

NopSCADlib ships actual board outlines. To verify the Pi 5 + fan + cells fit under the cab and inside the boiler:

```
include <NopSCADlib/lib.scad>
include <NopSCADlib/vitamins/pcbs.scad>
translate([cab_x, 0, footplate_z]) pcb(RPI4);   // or the Pi 5 model if present
```

Place these *ghost* vitamins in a verification-only assembly (not the printed output), then run the `tools/collision/` overlap gate between the shell interior and the vitamin bounding solids. This catches "the Pi's USB stack collides with the cab wall" before printing - the SPEC's "MEASURE the assembled Pi+battery cage" requirement, done in CAD.

13.4 trimesh penetration math (the overlap gate internals)

The repo's overlap gate computes signed-volume of pairwise boolean intersections. Conceptually:

```
import trimesh
a = trimesh.load('shell.stl'); b = trimesh.load('pi_solid.stl')
inter = a.intersection(b)            # manifold/blender backend
vol = inter.volume if not inter.is_empty else 0.0
FAIL = vol > EPS                     # EPS ~ 8 mm^3 (sliver/numeric noise)
```

For *intended* contacts (clip in socket), whitelist the pair or subtract the intended mating volume first. The signed-tetrahedron volume sum (over each triangle fan to the origin) is the robust way to get a watertight mesh's volume without relying on the boolean backend's own volume report.

13.5 Depth Anything V2 / SAM 2 integration sketch

The one-shot pre-processor that turns the reference photo into the loop's inputs:

```
# 1) SAM 2: click the loco -> binary silhouette
#    predictor.set_image(ref); masks = predictor.predict(point_coords=[[x,y]], ...)
#    save the loco mask as mask_ref.png (then ImageMagick-normalize to render frame)

# 2) Depth Anything V2: relative depth of the whole photo
#    depth = depth_model.infer_image(ref)   # HxW float, relative
#    depth_loco = depth * mask              # depth of just the loco
#    save normalized depth as a second critic channel (front-to-back ordering)
```

These run once at the start; the loop itself never re-invokes them. The mask becomes the fixed `mask_ref` target; the depth is an optional extra image handed to the critic so it can reason about the *depth* axis the side silhouette can't show (e.g., "the running boards stick out this far in front of the boiler").

13.6 Differentiable rendering, if you ever go gradient-based

Should the gradient-free loop prove too slow (it won't for a few-dozen parameters, but for a few-hundred it might), the gradient-based path is: reimplement the parametric geometry in PyTorch3D or a Mitsuba 3 scene with the matchable parameters as differentiable tensors, render the silhouette with a soft/analytic-AA renderer (nvdiffrast for speed, Mitsuba for physical accuracy), define `loss = 1 - IoU(render_sil, ref_sil)` (or a soft silhouette L2), and `loss.backward()` into the parameters. The hard part is always the **visibility discontinuity** at silhouette edges, which is exactly what nvdiffrast's multisample analytic AA and SoftRas's probabilistic coverage solve. This is a real engineering project (you give up OpenSCAD and the repo's gates operate on the exported mesh, not the generator), which is why it is the *last* resort, not the first.

---

14. Appendix B: anticipated objections and responses

A few predictable pushbacks, addressed, because they shape whether the recommendation sticks.

"Why not just let an image-to-3D model (Hunyuan3D/TRELLIS) make the loco and print that?" Because the output is a dense, non-parametric mesh with no LEGO studs, no controlled wall thickness, no guarantee of being watertight, and no way to swap a battery or tune clutch fit. It is a *sculpture*, not an *engineered part*. The project's requirements (LEGO-compatible interfaces, tool-free disassembly, internal electronics bay, calibrated clutch) are all parametric/CSG concerns the AI mesh cannot express. The AI mesh is useful as a proportion reference; it is not the deliverable. (Section 6.3.)

"Won't the LLM just write the whole OpenSCAD file and match the photo in one shot?" No - and the FlipFlop literature explains why letting it self-assess fails. An LLM can draft a plausible loco `.scad`, but the *pixel-perfect* match requires many small, verified adjustments, and without an external numeric metric gating acceptance, the model oscillates and degrades. The value is in the *loop with a metric*, not in a one-shot generation. (Sections 3.1-3.2.)

"Is silhouette IoU enough? It ignores depth and surface detail." For a flat side-elevation reference, the silhouette captures almost all the proportional information that matters (funnel/boiler/cab placement and size). Depth is added as a secondary channel via Depth Anything V2; surface detail (beading, rivets) is below the resolution at which the match matters and is added by the engineer as styling, not by the loss. If a second reference angle exists (e.g., a three-quarter view), add it as a second silhouette term and the IoU becomes multi-view. (Sections 5.1, 7.5.)

"Why ImageMagick and not a proper Python vision stack?" Because it is already installed, scriptable, deterministic, and fast, and the metrics it provides (AE, IoU via set ops, DSSIM) are exactly the silhouette measures the loop needs. A heavier stack (OpenCV, scikit-image) is fine if you already use it, but adds no capability the loop requires. Keep the measurement apparatus minimal and boring - it must be trustworthy above all. (Sections 5.6, 9.7.)

"BOSL2 is huge - won't it slow renders?" BOSL2 is large as *source*, but you only pay for what you call, and its rounded primitives are *cheaper* than the `minkowski()`/`hull()` hacks they replace. With the manifold boolean backend the assembly renders fast. The library size is a one-time include cost, not a per-render one. (Sections 2.1, 9.1.)

This concludes the report. The companion `sources.md` lists every cited source with a one-line annotation; the recommended next action is item 1 of Section 8.2 - lock the camera and build `match/score.sh` - because every other step depends on a reproducible render and a numeric metric.

---

15. Appendix C: the algorithm theory, expanded

Section 5 introduced the algorithms; this appendix gives the working-engineer's depth on the ones that matter most for this pipeline, with the math made concrete.

15.1 Silhouette IoU as an optimization objective - properties

Treat the rendered silhouette `S(theta)` (a binary mask, function of the parameter vector theta) and the fixed reference silhouette `R`. Define:

```
IoU(theta) = |S(theta) AND R| / |S(theta) OR R|
loss(theta) = 1 - IoU(theta)
```

Properties that make this a good objective for the LLM loop:

- **Bounded and interpretable.** IoU is in [0,1]; "IoU = 0.92" is a number an engineer and a critic both understand, unlike an unbounded pixel sum whose scale depends on image size.
- **Translation/scale-sensitive in the right way.** Because the camera is locked and both masks are in the same frame, a too-large boiler reduces IoU (union grows faster than intersection); a shifted funnel reduces it (intersection shrinks). The metric responds to exactly the errors you want corrected.
- **Plateau-prone, hence coarse-to-fine.** Far from the optimum, many single-parameter moves barely change IoU (the masks barely overlap), so gradients-by-finite-difference are flat. Coarse-to-fine ordering (match the bounding box first) ensures early moves land you in the basin where per-parameter IoU changes are large and informative.
- **Non-convex.** There can be local optima (a funnel matched to the wrong dome, say). The unfrozen polish round (Section 9.5) and the changelog (which discourages re-trying failed moves) are the practical escapes.

15.2 Per-feature decomposition

A single global IoU conflates all errors. Using the SAM sub-masks (funnel, boiler, cab) you can compute *per-feature* IoU and hand the critic a vector:

```
IoU_funnel, IoU_boiler, IoU_cab, IoU_pilot, ...
```

This tells the critic *which* feature is worst, sharpening its ranked-delta output ("boiler IoU 0.95 but funnel IoU 0.62 -> fix the funnel"). It also lets the orchestrator weight features (a wrong funnel is more visually salient than a slightly-short running board) by combining into a weighted loss `sum_i w_i (1 - IoU_i)`. This is the practical upgrade from one scalar to a structured objective, and it maps directly onto the coarse-to-fine phases.

15.3 Why inverse procedural modeling fits this problem exactly

IPM's defining feature is that the search space is *low-dimensional and meaningful*: a few dozen named parameters (boiler length, funnel fraction, cab height), each with a physical interpretation and a sane range. Contrast with general single-image-to-3D, which searches a *high-dimensional* mesh-vertex or latent space with no per-dimension meaning. The low-dimensional, interpretable space is what makes:

- **Gradient-free search tractable** - a few dozen parameters is well within reach of a propose-evaluate-accept loop; a few thousand vertices is not.
- **The critic's job easy** - it reasons about "funnel too tall," a named feature, not about which of 10,000 vertices to nudge.
- **The output editable forever** - the engineer can later tweak `funnel_height` by hand; a fitted mesh has no such handle.

The MDPI single-view IPM paper formalizes the gradient-based version; the LLM loop is the gradient-free version of the same idea, trading per-step speed for not needing a differentiable generator. For a 3D-printable, parametric, LEGO-interfaced part, the gradient-free IPM loop is not a compromise - it is the *correct* formulation, because the deliverable must be parametric anyway.

15.4 The role of depth and multi-view, formally

A single silhouette `R` under-determines the 3D model: many 3D shapes project to the same side outline (the depth axis is free). Three ways to add constraints:

1. **Engineering priors** - the SPEC already fixes the depth axis (7-stud width, boiler OD 60, etc.). These are hard constraints baked into `constants.scad`, removing the depth ambiguity by fiat. This is the primary resolution for this project.
2. **Monocular depth** (Depth Anything V2) - a soft, relative prior on the depth ordering, used as a critic channel, not a hard constraint (it's not metric).
3. **A second reference view** - if a three-quarter or front photo exists, add its silhouette as a second IoU term. Two views from different azimuths strongly constrain the 3D shape (this is the limit toward photogrammetry). The loss becomes `(1 - IoU_side) + lambda (1 - IoU_front)`, optimized jointly.

For the loco, (1) dominates - the width and internal dimensions are dictated by the LEGO grid and the electronics, so the silhouette match is really a 2.5D problem: get the side profile right, and the depth is already pinned by the spec. That is a much easier problem than general single-image 3D, and it is why this pipeline is tractable with simple tools.

15.5 Marching cubes and the AI-mesh bridge, concretely

If you *do* generate an AI reference mesh (TRELLIS/Shap-E) to overlay, the bridge is: the implicit models (Shap-E) output a function `f(x,y,z)`; marching cubes polygonizes `f = 0` into a triangle mesh; you import that mesh, orient it side-on in the same frame as your OpenSCAD render, render *its* silhouette, and add it as an auxiliary overlay the critic can consult ("the AI mesh suggests the cab roof slopes back"). The mesh's own quality (non-watertight, dense) doesn't matter because you only use its *silhouette*, which is robust to mesh defects. This is the safe way to use a powerful-but-unprintable AI output: extract the one robust signal (outline) and discard the rest.

---

16. Appendix D: decision matrix - which tool for which sub-task

A compact lookup tying each concrete sub-task of this project to the recommended tool and the fallback.

Sub-task	Primary tool	Fallback / escalation
Author the shell geometry	OpenSCAD + BOSL2	SolidPython2 (for sweeps); libfive (organic blends)
LEGO stud/anti-stud interfaces	cfinke/LEGO.scad modules in constants.scad	brickify (non-rect base); real-LEGO inserts
Fillets/chamfers	BOSL2 `rounding=/edge_mask`	minkowski() (slow); libfive SDF blend
Funnel flare / beading sweep	BOSL2 `path_sweep/skin`	dotSCAD splines
Render for match loop	openscad MCP / preview.sh –render	F3D (cross-check)
Reference silhouette	SAM 2	ImageMagick threshold (clean bg)
Reference depth	Depth Anything V2	(skip; spec pins depth)
Proportion scaffolding mesh	TRELLIS / Hunyuan3D	InstantMesh; skip entirely
Match metric	ImageMagick AE + IoU	OpenCV/scikit-image
Manifold gate	OpenSCAD –render grep + trimesh	Open3D decomposition; ADMesh
Overlap/penetration	tools/collision/ + trimesh	manifold boolean directly
Printability check	fdm-printability skill + Bambu CLI slice	Netfabb (manual final pass)
Internal fit (Pi/fan/cells)	NopSCADlib vitamins + overlap gate	hand-measured bounding cubes

Sub-task	Primary tool	Fallback / escalation
Mesh repair (if AI import)	pymeshlab / ADMesh	Netfabb / Meshmixer (GUI)
Thermal bay sanity	Elmer (steady-state)	hand calc (conduction estimate)
Clip/cab structural sanity	CalculiX via FreeCAD FEM	hand calc (beam bending)
2D dimensioned drawings	projection(cut=true) + DXF + matplotlib	manual annotation
Final slice + G-code	Bambu Studio CLI	PrusaSlicer/Orca CLI
Delivery (previews + STL)	tg –photo / –file (repo protocol)	Tailscale file cp

The pattern across the matrix: a small, deterministic, scriptable primary tool for each task, with a heavier or GUI fallback only for the rare hard case (a tricky AI-mesh import, a final manual repair). That is the shape of a maintainable, automatable pipeline - and it is almost entirely buildable on what this repo already has, plus BOSL2, a SAM 2 pre-pass, and three small match-loop scripts.

---

17. Appendix E: concrete critic-prompt templates

The methodology lives or dies on how the vision critic is prompted. These are ready-to-adapt templates implementing the patterns from Section 3.3.

17.1 The per-round critic prompt

```
You are a CAD silhouette critic. You see an OVERLAY image: the REFERENCE locomotive
outline in RED, the current RENDER outline in CYAN, matched regions in GREY. You also
see an EDGE-OVERLAY (red=reference edges, cyan=render edges) and a CHANGELOG of prior
edits with their score impact.

Current silhouette IoU: {iou}.  Target IoU: {target}.

CHANGELOG (do NOT propose a move already marked 'reverted'):
{changelog}

TASK: Propose exactly ONE parameter edit that will most increase IoU. Output strict JSON:
{
  "feature": "<which part: funnel|boiler|cab|pilot|dome_front|dome_rear|runningboard>",
  "param":   "<exact name in constants.scad>",
  "current": <number>,
  "target":  <number>,
  "delta_mm": <signed number>,
  "confidence": <0..1>,
  "reason": "<where the cyan/red mismatch is, in one sentence>"
}
If the residual is below target AND you have no high-confidence edit, output exactly:
{"CONVERGED": true}

Rules:
- Reason ONLY from where CYAN sticks out beyond RED (render too big/tall there) or RED
  beyond CYAN (render too small/short there).
- ONE parameter only. No coupled edits.
- Numbers in millimetres (or the fraction for *_frac params), with units implied by the param.
```

17.2 The orchestrator's accept/reject logic (pseudocode)

```
best = -1.0; changelog = []; stale = 0
while stale < MAX_STALE:
    crit = call_vision_critic(overlay, edge_overlay, changelog, best, TARGET)
    if crit.get("CONVERGED"): break
    old = read_param(crit["param"])
    write_param(crit["param"], crit["target"])          # apply ONE edit
    if not renders_ok():                                 # ReLook zero-reward anchor
        write_param(crit["param"], old)
        changelog.append(f'{crit["param"]} {old}->{crit["target"]}: INVALID RENDER reverted')
        stale += 1; continue
    render_silhouette(); score = iou()                   # measure
    gates_ok = manifold() and consistency() and printability()
    if score > best + MARGIN and gates_ok:               # forced-monotonic accept
        changelog.append(f'{crit["param"]} {old}->{crit["target"]}: IoU {best:.3f}->{score:.3f} OK')
        best = score; stale = 0
    else:
        write_param(crit["param"], old)                  # revert
        tag = "gate FAIL" if not gates_ok else "no improve"
        changelog.append(f'{crit["param"]} {old}->{crit["target"]}: IoU {best:.3f}->{score:.3f} {tag} rev
        stale += 1
return best, changelog
```

Three things make this robust and worth restating: (1) **the metric, not the model, decides acceptance** - the critic only proposes; (2) **invalid renders score worst-possible** - no reward hacking; (3) **the changelog is fed back** - the critic never amnesiacally re-tries a reverted move, which is the concrete antidote to the FlipFlop effect. The `MARGIN` prevents accepting noise-level "improvements"; `MAX_STALE` is the ReLook resample cap that ends a plateau.

17.3 Coarse-to-fine phase controller

Wrap the loop in a phase controller that freezes parameter subsets:

```
PHASES = [
    ["total_length", "total_height", "baseline_z"],                  # bounding box
    ["boiler_len", "boiler_d", "cab_len", "cab_h", "smokebox_len"], # major masses
    ["funnel_frac", "funnel_h", "funnel_flare", "dome_f_frac",       # landmarks
     "dome_r_frac", "dome_d", "pilot_h", "buffer_z"],
    ["cab_window_w", "cab_window_h", "beading_r", "headlight_x"],    # fine details
]
for phase_params in PHASES:
    freeze_all_except(phase_params)
    best, log = match_loop()          # the loop from 17.2, restricted to phase_params
# final unfrozen polish round to escape ordering artifacts (Section 9.5)
unfreeze_all(); match_loop()
```

This is the full controller: phases gate *which* parameters the critic may touch (by listing only those in the prompt's allowed set), the loop enforces monotone acceptance within a phase, and a final free round polishes. It is perhaps 150 lines of glue around tools the repo already has, and it is the entire "AI-assisted pixel-perfect" machine.

17.4 Failure-signature playbook

When the loop misbehaves, the signature usually points to one cause:

Symptom	Likely cause	Fix
Score never improves, critic keeps proposing	masks mis-registered (scale/crop)	re-lock camera, force same `WxH!`, re-threshold
Loop "converges" instantly at low IoU	blank/invalid render scored as match	enforce IoU=0 on all-background mask (zero-reward)
Oscillation despite monotonic rule	MARGIN too small (noise accepted)	raise MARGIN above per-render noise floor
Critic edits the wrong feature	global IoU only, no per-feature signal	add SAM sub-mask per-feature IoU vector
Good side match, wrong depth/proportion	single view under-constrains depth	rely on spec-pinned dims; add second view if available
Edits help IoU but break printability	gate not wired into accept	add manifold/printability to the AND in accept

Most "the AI loop doesn't work" reports trace to the first two rows - a measurement bug, not a model bug. The measurement apparatus (camera lock + mask registration + zero-reward on invalid) must be correct *before* you trust any critic feedback. That is the single most important operational takeaway of this entire report: **fix the metric before you tune the model.**

---

18. Appendix F: concrete install/run specifics per tool

The survey sections deliberately stayed conceptual. This appendix gives the practical "what does it cost to run" facts an engineer needs before adopting each tool: install command, license, and (for the AI models) approximate hardware/runtime. Figures are order-of-magnitude from the projects' own docs and the comparison sources cited earlier.

18.1 Authoring and mesh tools

Tool	Install	License	Notes
OpenSCAD	`brew install openscad` (macOS); apt/AppImage on Linux	GPL-2.0	nightly builds have the manifold fast-CSG backend
BOSL2	clone into `~/.local/share/OpenSCAD/libraries/`	BSD-2	header-only `.scad`; `include <BOSL2/std.scad>`
CadQuery	`pip install cadquery` (or conda)	Apache-2.0	pulls OCP/OpenCASCADE wheels
build123d	`pip install build123d`	Apache-2.0	same OCCT backend; needs Python 3.10+
SolidPython2	`pip install solidpython2`	LGPL-2.1	emits `.scad`; pair with the openscad binary
Blender	`brew install --cask blender`	GPL	`bpy` also `pip install bpy` for headless module

Tool	Install	License	Notes
trimesh	`pip install "trimesh[easy]"`	MIT	`[easy]` pulls manifold/rtree/shapely backends
Open3D	`pip install open3d`	MIT	prebuilt wheels; large (~300 MB)
PyVista	`pip install pyvista`	MIT	needs VTK; off-screen via `pv.OFF_SCREEN=True`
pymeshlab	`pip install pymeshlab`	GPL-3.0	bundles MeshLab filters
manifold	`pip install manifold3d`	Apache-2.0	the boolean kernel as a wheel
ADMesh	`brew install admesh` / apt	GPL-2.0	tiny C binary
F3D	`brew install f3d`	BSD-3	VTK viewer + headless `--output`
Gmsh	`pip install gmsh` or `brew install gmsh`	GPL-2.0	Python API + `.geo` DSL
ImageMagick	`brew install imagemagick`	ImageMagick (Apache-like)	the `magick`/`compare` binaries

For this project the minimal new install footprint is small: BOSL2 (a clone), ImageMagick (likely already present), and `pip install trimesh manifold3d` for the overlap math. Everything else is optional or already in the repo.

18.2 Slicers and FEA

Tool	Install	License	Notes
Bambu Studio	download app; CLI is the bundled binary	AGPL-3.0	A1 profiles ship with it; headless `--slice`
PrusaSlicer	`brew install --cask prusaslicer`	AGPL-3.0	`prusa-slicer` CLI
CuraEngine	build from source / conda	AGPL-3.0	headless core, no GUI
CalculiX (ccx)	`brew install calculix-ccx` / apt	GPL-2.0	`.inp` in, `.frd` out
Elmer	download / apt `elmerfem-csc`	GPL	multiphysics; thermal
FreeCAD	`brew install --cask freecad`	LGPL-2.0	`freecadcmd` headless + FEM workbench

18.3 AI perception/3D models - hardware and runtime

These are the heavy dependencies; know the cost before committing. Figures are approximate, from the projects' docs and the comparison sources in Section 6.

Model	Install	VRAM (approx)	Runtime	License	Role here
SAM 2	`pip install` from Meta repo + checkpoint	4-8 GB	<1 s/image (GPU)	Apache-2.0	reference mask (high value)
Depth Anything V2	pip + HF checkpoint (S/B/L)	2-6 GB	~0.1-1 s/image	Apache-2.0 (code)	proportional depth (once)

Model	Install	VRAM (approx)	Runtime	License	Role here
TripoSR	HF repo + weights	~6 GB	<1 s/image	MIT	fast rough mesh (scaffold)
Stable Fast 3D	HF repo + weights	~7 GB	~1 s/image	community (non-commercial nuance)	fast textured mesh
InstantMesh	HF repo + weights	~16 GB	seconds	Apache-2.0	mid-tier mesh
Hunyuan3D 2.x	Tencent HF repo + weights	~16-24 GB	tens of s	Tencent community	high-quality mesh
TRELLIS / .2	Microsoft repo + weights	~16 GB	~3 s/512^3 (H100)	MIT	quality leader (scaffold)
Point-E	`pip` from OpenAI repo	~4 GB	seconds	MIT	coarse point cloud
Shap-E	`pip` from OpenAI repo	~6 GB	seconds	MIT	implicit -> marching cubes

Practical reading: **SAM 2 and Depth Anything V2 are the only AI models worth standing up for this project**, and both are light (a few GB VRAM, sub-second) and permissively licensed - a one-time pre-pass on the single reference photo. The image-to-3D models are heavier (16-24 GB VRAM for the good ones) and only justify the setup if you decide the multi-angle scaffolding mesh meaningfully speeds the first coarse pass; for a single clean side reference, you likely skip them entirely. Note the **license nuance**: several image-to-3D weights carry non-commercial or community licenses (Hunyuan3D's Tencent license, Stable Fast 3D's community terms) - fine for a personal hobby loco, but read them before any commercial use. SAM 2, Depth Anything V2 (code), TripoSR, TRELLIS, Point-E, and Shap-E are the permissive (Apache/MIT) ones.

18.4 A minimal viable setup for THIS repo

To run the recommended pipeline (Section 8) with the least new machinery:

```
# 1. OpenSCAD authoring + BOSL2 (the only authoring additions)
git clone https://github.com/BelfrySCAD/BOSL2 \
    "$HOME/.local/share/OpenSCAD/libraries/BOSL2"
# LEGO interface modules: lift from a clone, don't depend at render time
git clone https://github.com/cfinke/LEGO.scad /tmp/LEGOscad   # copy stud()/anti_stud()

# 2. Match-loop measurement (the load-bearing additions)
brew install imagemagick           # AE/IoU/overlay metrics
pip install trimesh manifold3d     # overlap + manifold math for gates

# 3. (optional, one-shot) reference pre-processing
#    SAM 2 + Depth Anything V2 from their HF repos, run once on the reference photo

# 4. Final printability oracle: Bambu Studio (already needed to print)
```

That is the whole adoption cost: one library clone, two pip packages, and ImageMagick - on top of the OpenSCAD/MCP/gate infrastructure the repo already has. The AI scaffolding and FEA are strictly optional escalations. This is the concrete, minimal answer to "what do I install to start" - and it underlines the report's thesis: the pixel-perfect loop is mostly *method* (camera lock + metric + forced-monotonic acceptance), not heavyweight new tooling.